

MSc in Cybersecurity Engineering

Embedded Operating Systems and Architecture

01WMQWQ · A.Y. 2026/27

Professor: Stefano Di Carlo

Notes by Emanuele Tocci

Personal study notes, not official course material

Contents

1. Toolchain
2. Boot CPU Config
3. Device Interrupts Syscalls

Generated on 18/10/2026

1 Operating Systems and Kernel Architectures

1.1 Definitions and Purpose of an Operating System

Definition	Emphasis
Intermediary	**Middle layer** between hardware and applications that hides hardware details
Resource manager (IBM)	**Software** that allocates CPU, memory, I/O devices, and file storage among applications
Master control program (NIST)	First software loaded at power-on; its core, the kernel, stays resident in memory

> **IMPORTANT**
>
> An operating system (OS) **mediates** between application **software** and **hardware**: it gives controlled access to processors, memory, devices, and time, and decides how competing work is scheduled.

- **Purpose 1: make hardware usable.** Without an OS, every application must initialise the CPU, program timers, share communication links, and recover from interrupts on its own. The OS supplies rules and mechanisms so applications request services instead of re-implementing hardware policy.

```
application intent --> OS policy and mechanisms --> hardware actions
```

- **Purpose 2: manage finite shared resources**, even on a small board:
  - **CPU time**: which ready task runs now.
  - **Memory**: which code and data occupy RAM and Flash.
  - **Devices**: which component may configure a timer or consume a received byte.
  - **Events**: how an interrupt becomes work for the right task.

> **IMPORTANT**
>
> The OS boundary is a design choice, not a fixed feature list: on a microcontroller an OS may provide only task scheduling, synchronisation, and time services, while device drivers remain application-owned.

1.2 Monolithic, Microkernel, and Flat Designs

The classification describes **where OS services execute** and **how strongly they are isolated**.

Shape	Where services run	Advantage	Cost	Examples
Monolithic	Together in one privileged kernel address space	Direct, fast calls between services	Large trusted core	Linux
Microkernel	Minimal kernel; other services as separate servers	Small trusted base, fault containment	IPC overhead, more interfaces to design	seL4, QNX

Shape	Where services run	Advantage	Cost	Examples
Flat	Together in one firmware image	Tiny and direct	Little or no isolation	Bare-metal firmware

- **Monolithic kernel**: scheduling, drivers, filesystems, and networking share one kernel address space.
  - Services call each other directly and **quickly**.
  - A **faulty driver can corrupt kernel state**.
  - Practical when the feature set is broad and tightly coupled.
  - On a protected machine, applications execute outside the trusted kernel address space.
- **Microkernel**: the kernel keeps only **fundamental mechanisms**, usually scheduling, address-space management, and inter-process communication (IPC).
  - Drivers and other services are separate programs that exchange messages.
  - A failed service can be contained or restarted.
  - More interfaces and message paths must be designed correctly.
- **Flat firmware**: **one linked image** and **one physical address space**.
  - Startup code, kernel, drivers, and application tasks are all in the same Flash image and can all address the same RAM.
  - Tasks cooperate through shared memory and the scheduler.
  - No process loader, no virtual-memory translation, usually no hardware-enforced boundary between tasks.

> **IMPORTANT**
>
> Choosing a kernel shape The categories describe trade-offs in isolation, fault containment, overhead, and complexity. The relevant design question is not "which shape is best?" but "what must this system guarantee?".

1.3 Mechanisms of a Minimal OS and RTOS Kernel Services

A minimal flat OS is built from a chain of mechanisms, each depending on the previous one:



A real-time operating system (RTOS) such as FreeRTOS provides established implementations of these mechanisms:

- Task creation and priorities.
- Delays and timer-driven scheduling.
- Queues, semaphores, and mutexes.
- Interrupt-to-task communication.

FreeRTOS is a small kernel with real scheduling policies, but by default it has **no memory protection between tasks**: tasks still share one address space as in flat firmware. An RTOS API call is easier to reason about when the underlying state transitions are known.

2 Embedded Systems and Real-Time Constraints

2.1 Embedded Systems Versus General-Purpose Computers

An **embedded system** is a computer built into a larger device to **perform one dedicated job**, not a general-purpose machine on which arbitrary software is installed. Hardware and software (**firmware**) ship as **one unit**; the end user never sees a shell or a package manager.

- Examples: engine controllers, pacemakers, drones, washing machines, Wi-Fi routers, the microcontroller in a USB charger.

Embedded software **cannot rely on an OS** to create a process, assign an address space, discover devices, or provide standard input and output. It is built for a known processor and memory map: hardware assumptions are visible and non-negotiable, which does not make the software intrinsically simple.

Aspect	Embedded	General-purpose
Workload	One fixed task	Anything the user installs
Resources	KB of RAM and Flash	GB of RAM, TB of disk
Timing	Often **hard deadlines**	Best-effort
User model	No interactive user	Multi-user, multi-tasking
Cost and power	Cents, milliwatts	Dollars, tens of watts

2.2 Real-Time Means Bounded Response

Real-time concerns **bounded, predictable response**, not raw speed.

Requirement	Example
Functional	Sample a sensor correctly
Temporal	Sample it every 1 ms, before the deadline
Robustness	Continue safely when one input is missing

> ⚠ **WARNING**
>
> Correct value, wrong time A result computed after its deadline can be a failure even when its value is otherwise correct. A fast system is not necessarily a real-time system.

2.3 How Constraints Shape Embedded OS Design

Embedded OSes differ from desktop OSes because they are:

- **Resource-constrained**: kilobytes of RAM and Flash leave no room for a general-purpose kernel's overhead.
- **Real-time**: correctness often depends on *when* something happens, not only *whether* it happens.

- **Usually without user protection**: one vendor, one firmware image, one set of tasks; the isolation problems of a desktop OS mostly do not apply.
- An **embedded OS** is selected and **configured around a particular product**, not an unknown future workload:

General-purpose mechanism	Embedded replacement
Dynamic allocation	Static task counts and fixed buffers
General fairness policy	Simple priority scheduler
Layered device stack	Direct register access
Maximum throughput	Predictable execution

> **TIP**
>
> Small is not incomplete A small OS is complete when it matches the system's responsibilities.

2.4 Single Image, Single Failure Domain

Many MCU (Microcontroller Unit) products ship **one firmware image** containing startup code, drivers, OS, and application logic; **updating the image changes the whole system.** This simplifies deployment but raises the required discipline:

- A bad pointer can damage any writable state.
- A blocked high-priority task can stop an important service.
- Recovery and observability must be designed into the firmware.

Scheduling and synchronisation are the mechanisms that reduce these risks.

3 Microcontroller Architecture and the Cortex-M3 Machine Model

3.1 Microcontroller Versus General-Purpose CPU

A **microcontroller (MCU)** integrates on one chip what a desktop splits across a motherboard:

- **CPU core**: executes instructions (here an *ARM Cortex-M3*).
- **RAM**: volatile working memory, typically tens of KB.
- **Flash**: non-volatile program storage; code executes directly from it.
- **Peripherals**: timers, UART, GPIO, ADC, all on-chip.
- One chip, one clock, one address space: no external memory bus to wire, no BIOS, no disk.

Aspect	Desktop CPU	Microcontroller
Integration	Core only; RAM, storage, I/O on separate chips over buses	Core, memory, and peripherals on one chip
Address space	Per-process virtual address space via an MMU	One flat physical map for code, data, and peripheral registers
Memory protection	MMU	No MMU; at most an optional MPU on Cortex-M
Device access	Through a driver layer	Device registers are memory that software writes directly

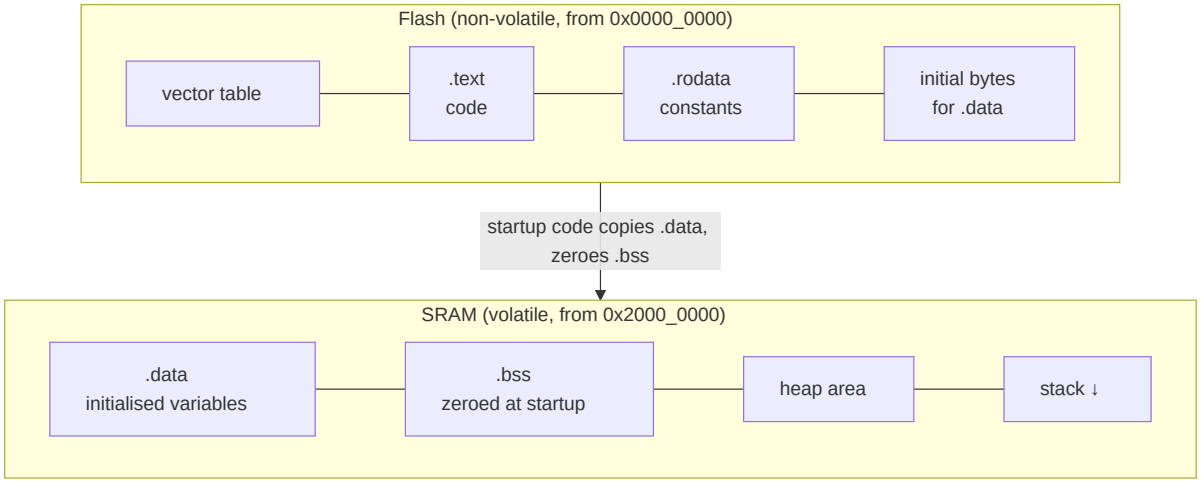
3.2 Registers and the Load/Store Instruction Set

Register role	Function
Program counter (`PC`)	Selects the **next instruction**
Stack pointer (`SP`)	Locates calls, local variables, and saved state
General registers	Hold operands and addresses
Status register	Records condition flags and interrupt state

- The Cortex-M3 has a **load/store** instruction set: arithmetic operates only on registers; loads bring a value from an address into a register, stores write it back. This means that every operation must be performed on a cpu register.
- When an exception occurs, the processor automatically saves essential state on the current stack so that execution can later resume.

3.3 Flash, SRAM, and the Physical Memory Map

- **Flash** keeps its contents without power; **SRAM** is fast writable memory whose contents are lost at reset or power loss.



- This split is the reason startup code must copy some bytes and clear others before C code can run safely (see the bare-metal runtime section).
- The CPU interprets an address according to the chip's memory map, not according to the C type used in the source.

Address range family	Typical meaning
`0x00000000...`	Code Flash, boot-visible memory
`0x20000000...`	SRAM
`0x40000000...`	Peripheral registers

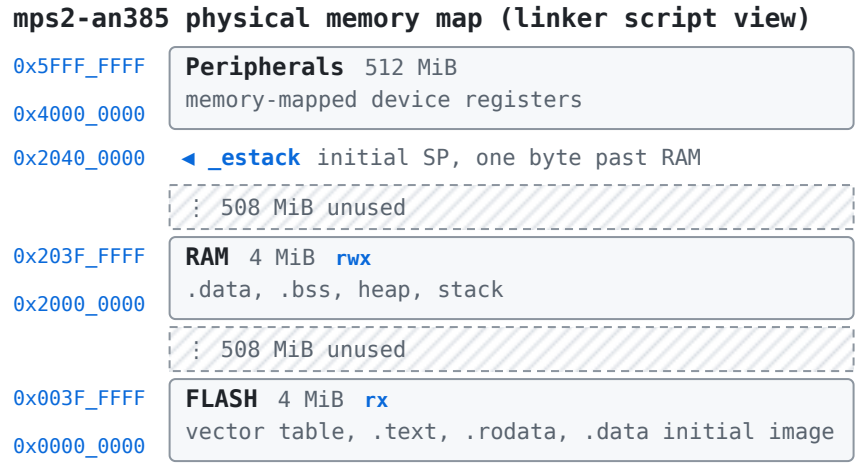
Exact sizes and addresses come from the board documentation and the linker script. For the QEMU `mps2-an385` machine (Cortex-M3), the linker script declares:

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

_estack = ORIGIN(RAM) + LENGTH(RAM);   /* = 0x20400000 */
```

- `FLASH (rx)` : readable and executable region at address `0x00000000` , 4 MiB.
- `RAM (rwx)` : readable, writable, executable region at `0x20000000` , 4 MiB.
- `_estack` : one byte past the last RAM byte, `0x20400000` ; the stack starts there and grows **downward** toward lower addresses.

The same map, drawn to scale of the populated regions:



This is a physical map, not a per-process virtual address space. No run-time loader chooses locations for a bare-metal image: all placement decisions are made at link time and recorded in the image.

3.4 Memory-Mapped Peripherals and `volatile`

Peripheral control registers sit at fixed addresses in the same address space. A driver accesses them with ordinary load and store instructions through a pointer to the documented register address.

Such accesses are declared `volatile`:

- A read may observe a changing hardware state.
- A write may trigger an external effect.
- The compiler therefore must not remove or merge the access as if it were an unused ordinary variable.

> ⚠ **WARNING**
>
> What `volatile` does not do `volatile` expresses the special observability of an access. It provides neither atomicity nor any general concurrency mechanism.

3.5 Cortex-M Reset Behaviour

After reset, the Cortex-M does not discover an operating system. The hardware reads two words from the **vector table** at the start of Flash:

- **Word 0**: the initial value of the main stack pointer.
- **Word 1**: the address of `Reset_Handler`.

Vector table read at reset (start of Flash)



`Reset_Handler` is the first software executed. It must establish the C runtime environment before calling `main()`.

Component	Role in the minimal image
CPU registers	Hold the current instruction address, stack pointer, and operands
Flash	Stores the vector table, code, constants, and the `.data` initial image
SRAM	Holds writable data, `.bss`, the heap reservation, and the stack
Peripherals	Expose registers in the same address space

4 Cross-Compilation Toolchain

4.1 Host, Target, and Emulator

Role	Meaning
Host	Computer running the editor, `make`, and the toolchain (x86-64 or Apple silicon)
Target	ARM Cortex-M3 machine for which the ELF is built
Emulator	QEMU model that executes target instructions on the host

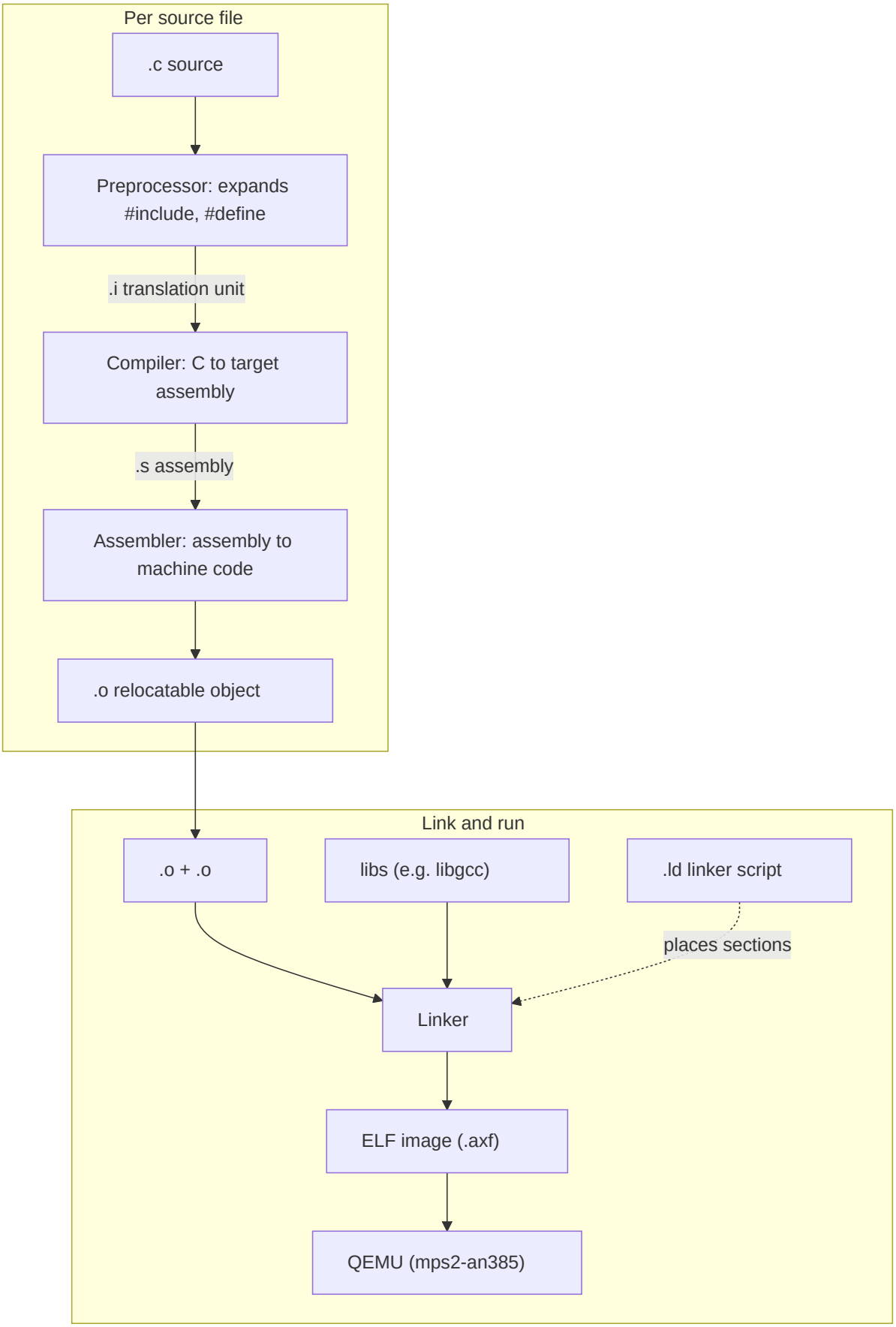
A native `gcc` emits code for the machine it runs on; it cannot produce Cortex-M3 firmware just because the source language is C. A **cross toolchain** (`arm-none-eabi-*`) runs on the host but emits code for the target instruction set architecture (ISA).

```
gcc -dumpmachine              # host triple, e.g. x86_64-linux-gnu
arm-none-eabi-gcc -dumpmachine  # target triple: arm-none-eabi
```

The triple `arm-none-eabi` means: ARM architecture, **no** operating system on the target, standard ARM **E**mbedded **ABI**.

- Consequences of "no OS":
  - No `_start` from a C library unless the application provides one.
  - No dynamic loader, and no process to return to when `main()` finishes.
  - No guaranteed `printf` console, no `malloc` backed by an OS.
- QEMU is not the target CPU, but it provides a reproducible board model before physical hardware is used.

4.2 Build Pipeline Stages


Per source file
.c source
Preprocessor: expands #include, #define
.i translation unit
Compiler: C to target assembly
.s assembly
Assembler: assembly to machine code
.o relocatable object
Link and run
.o + .o
libs (e.g. libgcc)
.ld linker script
places sections
Linker
ELF image (.axf)
QEMU (mps2-an385)

Stage	What it does	Stop with	Output
Preprocessing	Expands `#include`, macros, conditional compilation; pure text substitution	`-E`	`.i` translation unit
Compilation	Checks syntax and types, **optimises**, emits **ARM assembly** for the selected CPU and ABI	`-S`	`.s`
Assembly	Encodes assembly into a **relocatable object file**	`-c`	`.o`
Linking	Combines objects, resolves symbols, assigns final addresses	(full run)	ELF image

- `arm-none-eabi-gcc` is a driver that invokes each stage in turn; `make` decides which files need rebuilding.
- **Preprocessing** produces one **translation unit**: a single file of plain C with every include and macro expanded.

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -E src/main.c -o build/main.i
```

In `main.i`, `#include <stdint.h>` is replaced inline by its typedefs; `main()` is unchanged text with comments stripped. `-E` only substitutes text and does not touch program logic.

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -S src/main.c -o build/main.s
```

In `main.s`, the `bkpt 0xab` instruction from the inline assembly in `main.c` passes through untouched; everything around it is Thumb-2 assembly generated from the C source.

4.3 Target Compiler and Linker Flags

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -std=gnu17
CFLAGS += -Wall -Wextra -O0 -g3

LDFLAGS := -T scripts/mps2_m3.ld
LDFLAGS += -Xlinker -Map=build/output.map
```

Flag	Meaning
`-mthumb -mcpu=cortex-m3`	Selects the Thumb instruction subset implemented by the Cortex-M3
`-nostartfiles`	Excludes the toolchain's default C startup objects, so reset handling is done by the project's own startup code
`-ffreestanding`	Compiles for a freestanding (bare-metal) C environment, without hosted-library assumptions
`-std=gnu17`	C17 with GNU extensions

Flag	Meaning
`-Wall -Wextra`	Warning policy
`-O0 -g3`	No optimisation and full debug information: a debuggable, source-corresponding image
`-T scripts/mps2_m3.ld`	Uses the given linker script to describe memory and section placement
`-Xlinker -Map=build/output.map`	Makes the linker emit a map file of every placement and symbol address
`-nostdlib -lgcc`	Links no standard library, only `libgcc` for compiler support routines

- Target options must describe the actual machine: instruction set and Thumb execution, floating-point assumptions, freestanding environment, warning and optimisation policy. Host compiler defaults are not safe substitutes.
- Make variables keep the policy in one place:

Variable	Role
`CC`	Chooses the cross compiler (`arm-none-eabi-gcc`)
`CFLAGS`	Governs compilation of each source
`LDFLAGS`	Governs memory layout and the final link

> **TIP**
>
> Diagnosing a build failure The full command line is the exact contract that produced the failure: copy it before changing anything.

- **Debug information** (`-g3`) is stored in the ELF for host tools: QEMU and a debugger can map an instruction address back to a C line, function, or variable. The target still executes only machine instructions; debug information does not change the boot image's behaviour.

4.4 Object Files, Symbols, and the Linker

An object file contains machine-code bytes, section names, symbol definitions, and **references whose final addresses are not yet known**. Each object is built independently, so its sections have no final position in Flash or SRAM.

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -c boot/startup.c -o build/boot/startup.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -c src/main.c -o build/src/main.o
arm-none-eabi-nm build/boot/startup.o build/src/main.o
```

`nm` output	Meaning
`U main` in `startup.o`	Undefined: referenced (by `Reset_Handler`) but not defined in this object

`nm` output	Meaning
`T main` in `main.o`	Defined, in the `.text` section

- Unresolved references are normal at this stage; neither object is a runnable program.
- The **linker**:
  - Concatenates matching sections from all objects.
  - Resolves every symbol (e.g., `main`, `_estack`) to a final address and patches all references.
  - Checks that every referenced symbol has exactly one valid definition.

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding \
    -T scripts/mps2_m3.ld -Xlinker -Map=build/output.map -nostdlib -lgcc \
    build/boot/startup.o build/src/main.o -o build/scratch-os.axf
arm-none-eabi-nm build/scratch-os.axf | grep -w main
```

- In the final image `main` is `T`: the `U` in `startup.o` has been connected to the `T` in `main.o`. The map file shows which object supplied `main` and its final address.
- The **linker script** (`.ld`) states where things go: physical addresses of `FLASH` and `RAM`, the contents of `.text`, `.data`, `.bss`, where the stack starts, and where the vector table must sit.
- On a desktop the default linker script is invisible; on bare metal it must be supplied, because only the developer knows the chip's memory map.
- Separate compilation followed by linking enables fast incremental builds.

5 ELF Image Format and Memory Layout

5.1 Sections, Segments, and the ELF Header

- ELF (Executable and Linkable Format) is the format of the final image `build/scratch-os.axf`; the `.axf` suffix conventionally denotes an **ARM executable** with debug information.
- The image is a 32-bit, little-endian ARM ELF executable with several views of the same bytes:
  - **ELF header** at offset 0: points to the section and program-header tables and names the entry point.
  - **Sections** (for the linker and debugger): `.text`, `.rodata`, `.data`, `.bss`, symbol tables, debug information.
  - **Program headers / segments** (for a loader): which chunks map to which memory at run time.
  - **Symbols**: names for final addresses.
- Inspection tools: `arm-none-eabi-readelf`, `arm-none-eabi-objdump`, `arm-none-eabi-nm`, `arm-none-eabi-size`.

Section	Contents	Placement
`.isr_vector`	Initial **stack pointer** and handler addresses	Start of Flash
`.text`	**Executable** instructions	Flash
`.rodata`	**Read-only** constants	Flash
`.data`	Writable **variables** with initial values	Runs in SRAM, initial image stored in Flash
`.bss`	Writable **zero-initialised** variables	SRAM

`arm-none-eabi-objdump -h` lists each section with two addresses:

- **VMA** (virtual memory address): the run address.
- **LMA** (load memory address): where the bytes are stored in the image.
- They differ only for `.data`: VMA in RAM, LMA in Flash. This difference is what the startup copy loop resolves.

5.2 Entry Point and the Thumb Bit

5.2.1 ARM and Thumb instruction sets

ARM cores define two instruction sets: **ARM** (32-bit instructions) and **Thumb** (16-bit instructions, extended to mixed 16/32-bit in Thumb-2). A core supporting both must know, at every branch, which set the target code uses.

> **IMPORTANT**
>
> The Cortex-M3 executes **only Thumb**.

Thumb instructions are 2 or 4 bytes long and always start at an **even** address. The linker places them in the `.text` output section, in Flash.

As a consequence, the bit 0 of the start address of an instruction is **always 0** and **carries no information**. ARM reuses it as a state flag in every **code pointer**, i.e. every stored address the CPU branches to (vector-table entry,

function pointer, ELF entry point, target of `bx` / `blx`):

- **Bit 0 = 1**: the target is **Thumb code**.
- **Bit 0 = 0**: the target is **ARM code**, which does not exist on Cortex-M and raises a **UsageFault** (invalid state).

5.2.2 How the linker produces `0x48` and `0x49`

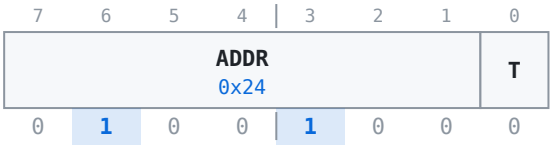
- **Placement**: the linker fills Flash in the order given by the linker script. The 64-byte vector table occupies `0x00` – `0x3F`; `.text` begins at `0x40` with the code of `startup.o`, inside which the first instruction of `Reset_Handler` lands at `0x48`. The symbol `Reset_Handler` therefore takes the value `0x48`.
- **Marking**: the compiler (or `.thumb_func` in assembly) records in the object file that `Reset_Handler` is a Thumb function.
- **Writing code pointers**: wherever the address of a Thumb function is stored as a branch target, the linker writes `address | 1` (bitwise OR: bit 0 set to 1, i.e. +1 for an even address):
  - Vector table word 1 (`.word Reset_Handler`): `0x48 | 1 = 0x49`.
  - Entry field of the ELF header (from `ENTRY(Reset_Handler)` in the linker script): `0x49`.
- **Reset**: the CPU loads `0x49` from vector word 1, checks bit 0 = 1 (Thumb, valid), sets the `T` bit of `xPSR`, clears bit 0, and starts executing with `PC = 0x48`.

```
0x48 = 0100 1000   location of the first instruction
0x49 = 0100 1001   code pointer: same location + Thumb flag in bit 0
```

The two values side by side: the address bits are identical, only bit 0 differs.

symbol 8-bit **= 0x48**

Low byte of Reset_Handler's address (map file, nm, objdump)



Bits	Field	Value	Description
7:1	ADDR	0x24 (36)	Instruction address bits
0	T	0 → even location	Bit 0 0 = even location, 1 = Thumb code pointer

pointer 8-bit **= 0x49**

Low byte of the code pointer (vector word 1, ELF entry)



Bits	Field	Value	Description
7:1	ADDR	0x24 (36)	Instruction address bits: the same instruction
0	T	1 → Thumb code pointer	Bit 0 0 = even location, 1 = Thumb code pointer

5.2.3 Inspection tools and what they report

Tool	What it is	Value for `Reset_Handler`
Map file `build/output.map`	Text report written by the linker during the link (`-Map=`): output sections, input pieces with their `.o` , address of every symbol	`0x00000048` (location)
`arm-none-eabi-nm`	Lists the symbols of an object or ELF file with address and type letter	`00000048 T Reset_Handler` (location)
`arm-none-eabi-objdump -d`	Disassembles the code: every instruction with the address where it is stored	`00000048 <Reset_Handler>:` (location)
`arm-none-eabi-readelf -h`	Prints the ELF header, including the entry point	`Entry point address: 0x49` (code pointer)

- Map file, `nm` , and `objdump` answer "where are the bytes stored?": even value.
- ELF entry point and vector table answer "how does the CPU branch there?": odd value.
- No instruction starts at `0x49` : the byte at `0x49` belongs to the instruction that starts at `0x48` . Both values identify the same instruction, once as a location and once as a branch target.
- Data pointers are unaffected: bit 0 has this meaning only for addresses used as branch targets (e.g., `&counter` is a plain address).

THUMB ADDRESS RULE

- Symbol address (where the bytes are stored): always even, e.g. `0x48` .
- Code-pointer value (how the CPU branches there): symbol address `| 1` , e.g. `0x49` .
- On Cortex-M, a code pointer with bit 0 = 0 raises a UsageFault.

5.3 Worked Layout from the Map File

5.3.1 What the map file is

The **map file** (`build/output.map` , also called *link map*) is a text report written by the linker while it builds the image, requested with `-Xlinker -Map=build/output.map` .

- It records every placement decision of the linker: where each piece of the program ended up in memory, how large it is, and which object file it came from.
- It is the reference for the question **"where did this function, variable, or constant end up?**".

```
Memory Configuration

Name             Origin             Length
FLASH            0x00000000         0x00400000
RAM              0x20000000         0x00400000

Linker script and memory map

.isr_vector     0x00000000       0x40
 .isr_vector    0x00000000       0x40 build/boot/startup.o

.text           0x00000040       0xa0
 .text          0x00000040       0x64 build/boot/startup.o
                0x00000048                Reset_Handler
 .text          0x000000a4       0x2c build/src/main.o
                0x000000bc                main
 .rodata        0x000000d0        0xe build/src/main.o
```

Line type	Example	Meaning
Memory Configuration row	`FLASH 0x00000000 0x00400000`	A `MEMORY` region of the linker script: name, start, length
Non-indented section line	`.text 0x00000040 0xa0`	An **output section** of the final image: start address and size
Indented line with a `.o`	`.text 0x00000040 0x64 build/boot/startup.o`	An **input piece** of that output section: address, size, source object file

Line type	Example	Meaning
Address followed by a name	`0x00000048 Reset_Handler`	A **symbol** (function or variable) and its final address

- The excerpt above is simplified; the real file also lists debug sections, discarded sections, and linker-script symbols.
- Static (file-local) functions and variables do not appear as symbols in the map file.

5.3.2 Placement rules that produce the layout

The linker fills each region following three ordering rules:

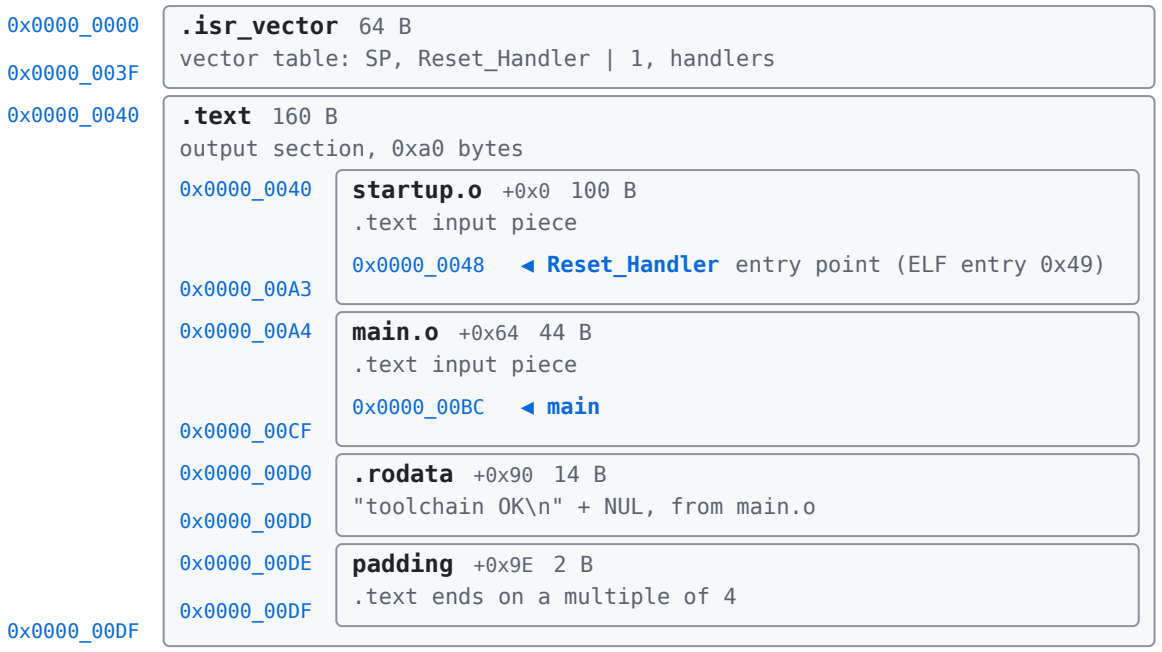
- **Order of output sections in the linker script**: `.isr_vector` first, then `.text`. The vector table (16 words × 4 bytes = 64 bytes = `0x40`) therefore starts at `0x00`, and `.text` starts right after it at `0x40`.
- **Order of object files on the link command line**: `startup.o` before `main.o`. The `0x64` bytes of `startup.o` occupy `0x40` – `0xA3`; the `0x2c` bytes of `main.o` follow at `0xA4` (`0x40 + 0x64`).
- **Order of rules inside `.text`**: `*(.text*)` (all code) before `*(.rodata*)` (all constants). The string literal therefore comes after **all** code, at `0xD0` (`0xA4 + 0x2C`), not next to `main`.

5.3.3 Flash Layout (`0x00` – `0xDF`, 224 bytes)

Address range	Size (bytes)	Content	Output section	From
`0x00` – `0x03`	4	Vector word 0: initial SP = `0x20400000`	`.isr_vector`	`startup.o`
`0x04` – `0x07`	4	Vector word 1: `Reset_Handler | 1` = `0x49`	`.isr_vector`	`startup.o`
`0x08` – `0x3F`	56	Vector words 2–15: addresses of the other exception handlers	`.isr_vector`	`startup.o`
`0x40` – `0x47`	8	Code of `startup.o` placed before `Reset_Handler`	`.text`	`startup.o`
`0x48` – `0xA3`	92	`Reset_Handler` (first instruction at `0x48`, the entry point) and the rest of the code of `startup.o`	`.text`	`startup.o`
`0xA4` – `0xBB`	24	Code of `main.o` placed before `main` (static helpers such as `semihost_write0`)	`.text`	`main.o`
`0xBC` – `0xCF`	20	`main`	`.text`	`main.o`
`0xD0` – `0xDD`	14	String literal `"toolchain OK\n"` plus terminating NUL	`.rodata`, folded into `.text`	`main.o`
`0xDE` – `0xDF`	2	Alignment padding (`.text` ends on a multiple of 4)	`.text`	linker

The same layout as a map, with the input pieces drawn inside their output section:

Flash after linking (from build/output.map)

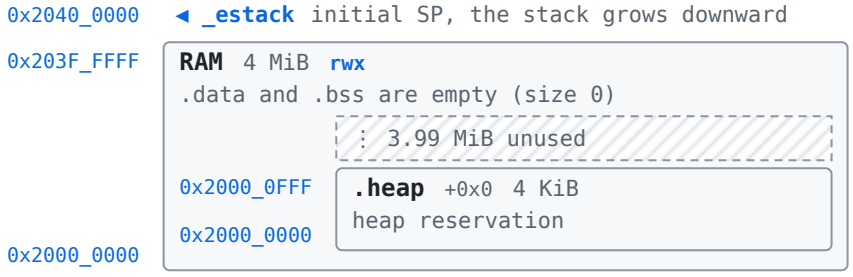


- End of `.text` : `0x40 + 0xA0 = 0xE0` , i.e. 224 bytes of Flash, the `text` value reported by `arm-none-eabi-size` .

5.3.4 RAM Layout (`0x20000000` – `0x203FFFFF` , 4 MiB)

Address range	Size (bytes)	Content	Output section
`0x20000000`	0	Initialised globals (none in this program)	`.data`
`0x20000000`	0	Zero-initialised globals (none in this program)	`.bss`
`0x20000000` – `0x20000FFF`	4096 (`0x1000`)	Heap reservation	`.heap`
`0x20001000` –…	—	Free RAM	—
…– `0x203FFFFF`	—	Stack, growing downward from `_estack` = `0x20400000`	—

RAM after linking



- `.data` and `.bss` have size 0, so they occupy no bytes and `.heap` starts at the same address.

> **EMPTY `.DATA` AND `.BSS`**
>
> - Size 0 only means that this program has no initialised or zero-initialised global variables.
> - The startup code must still copy `.data` and zero `.bss` in general: any global or static variable makes the same loops do visible work.

5.4 Flash and RAM Budgets

- `.bss` has a size in memory but usually no bytes in the ELF file: startup code creates its zero values.
- `.data` consumes **both** Flash (initial bytes) and SRAM (run-time copy).

```
Flash usage = code + constants + .data initial image
RAM usage   = .data + .bss + stack + heap reservation
```

`arm-none-eabi-size build/scratch-os.axf` reports `text` = 224, `data` = 0, `bss` = 4096 bytes:

- `text` = `0x40 + 0xa0` = 224 bytes (vector table plus code and constants).
- `bss` includes the 4096-byte heap reservation.
- `text + data` must fit in Flash; `data + bss` must fit in RAM: the same bytes of `.data` are counted against two budgets.

> **IMPORTANT**
>
> File size is not RAM usage The ELF file size includes debug information and excludes `.bss` ; memory budgets are read from the `size` columns and the link map.

6 Bare-Metal Runtime: Startup Code, Semihosting, and QEMU Execution

6.1 Startup Code Responsibilities

On a desktop, the OS loader and C runtime prepare memory and invoke the program. On a bare-metal target no such service exists at boot: the application supplies its own runtime.

- With `-nostartfiles`, the toolchain's startup objects are excluded and `boot/startup.c` takes over. The startup code must:
  - Provide the vector table.
  - Initialise `.data` (copy initial values from Flash to RAM) and `.bss` (fill with zeros).
  - Call constructors if the language or runtime requires them.
  - Call `main()` and define what happens if it returns.

After these steps, every global variable has its promised value when `main()` starts.

- `Default_Handler`: an **infinite loop** installed for unimplemented exception vectors. Stopping in a known loop is safer and more diagnosable than executing arbitrary memory after an unexpected exception.
- `main()` has **no operating system to return to**: if it returns, `Reset_Handler` enters a final `for( ;; ) {}` spin loop.

> ⚠ **WARNING**
>
> QEMU does not exit After `main()` returns, the target keeps executing the final spin loop, so QEMU keeps running and must be interrupted manually. The printed line is evidence that execution reached the intended point, not an exit notification.

6.2 Semihosting Output via `bkpt 0xab`

6.2.1 The problem

A bare-metal program has no console and no OS: `printf` needs a C library and an OS underneath (none present), and output through a UART needs a peripheral driver (not written yet).

- **Semihosting** solves this during development: the program running on the target asks the **host** (the computer running QEMU or a debugger) to perform an I/O operation on its behalf. The target borrows the host's console, files, and clock.

6.2.2 How a Semihosting Request Works

A semihosting request follows a fixed convention, defined by ARM's semihosting specification and shared by the program and by every debugger or emulator that supports semihosting. The convention states where the program places its request and which instruction announces it. Because both sides follow the same rules, the host knows exactly where to look for the request and how to interpret it.

Element	Role in the request	Value for string output
`r0`	Operation number: which service is requested	`0x04` (`SYS_WRITE0` , "print a NUL-terminated string")
`r1`	Argument: the data the service needs	Address of the string in target memory
`bkpt 0xab`	Signal: the request is ready	—

On the target side, the program loads `0x04` into `r0` , loads the address of the string into `r1` , and executes `bkpt 0xab` . On the host side, QEMU notices the breakpoint and recognises the immediate value `0xab` as a semihosting request. It reads `r0` to learn which service is requested and `r1` to find the string, copies the bytes from the target's memory up to the terminating NUL, and prints them on the host terminal. The target then resumes at the instruction following `bkpt` , as if that instruction had done nothing.

The choice of a breakpoint instruction is deliberate. `bkpt` normally halts the processor and hands control to an attached debugger. The semihosting convention reuses this behaviour, and the immediate value `0xab` tells the debugger or emulator that the halt is a service request, not a breakpoint set by a developer.

This pattern is a **trap**: the program suspends its own work, control passes to a more powerful entity outside the CPU that performs the operation, and control then returns to the program. A system call in an operating system has the same shape, with the kernel playing the role of the host.

> **WARNING**
>
> Development tool only Semihosting works only when a debugger, or an emulator started with semihosting enabled (`-semihosting` in QEMU), is attached and listening. On a real board with nothing attached, no one answers the request and `bkpt` simply halts the CPU. For this reason, production firmware produces its output through a peripheral, such as a UART driver, instead of semihosting.

6.2.3 Implementation in C

```c
#include <stdint.h>

static void semihost_write0( const char *message )
{
    register uint32_t    r0 __asm__( "r0" ) = 0x04;    /* SYS_WRITE0 */
    register const char *r1 __asm__( "r1" ) = message; /* string address */
    __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
}

int main( void )
{
    semihost_write0( "toolchain OK\n" );
    return 0;
}
```

Code	Meaning
`register uint32_t r0 __asm__( "r0" ) = 0x04;`	GCC extension: the variable is forced into register `r0` and holds the operation number `SYS_WRITE0`; standard C cannot choose registers
`register const char *r1 __asm__( "r1" ) = message;`	The string address is forced into register `r1`
`__asm__ volatile ( "bkpt 0xab" ... )`	Inline assembly (an assembly instruction embedded in C): inserts `bkpt 0xab`, which no C statement can express; `volatile` prevents the compiler from removing it
`: : "r"( r0 ), "r"( r1 )`	Input operands: guarantee that `r0` and `r1` hold their values when `bkpt` executes
`: "memory"`	Clobber: the instruction reads memory (the string), so all pending writes to memory are completed before it
`semihost_write0( "toolchain OK\n" );`	Prints the message on the host terminal: at C level the function means "the host prints `message`"

- `printf` is not used: it requires the C-library initialisation (newlib's semihosting-backed `stdio`) that the custom reset path does not perform.
- `arm-none-eabi-objdump -d build/scratch-os.axf | grep -A2 bkpt` shows the `bkpt 0xab` instruction inside the compiled image.

6.3 QEMU Board Model

```
qemu-system-arm -machine mps2-an385 -kernel build/scratch-os.axf -nographic -semihosting
```

- `qemu-system-arm` emulates an ARM system and selected board peripherals.
- `-machine mps2-an385`: the machine choice determines CPU, memory map, and available devices (here a Cortex-M3 board); it must match the memory model in the linker script.
- `-kernel`: the image to boot, which remains an ARM ELF, not a host (Linux or macOS) program.
- `-nographic`: target console routed to the host terminal.
- `-semihosting`: enables interception of `bkpt 0xab` requests.

6.4 What the Output Proves and Failure Localisation

Seeing `toolchain OK` proves that several independent contracts agree:

- The cross compiler emitted valid target instructions.
- The linker placed startup and application code at usable addresses.
- The processor found the vector table and reset reached `Reset_Handler`.
- Startup code established the runtime state and reached `main()`.
- The output mechanism (semihosting) was enabled and recognised by QEMU.

It does not prove that future drivers or timing properties are correct: it is a narrow but valuable check.

When QEMU prints nothing, work outward from the earliest guaranteed event:

- Confirm the ELF was rebuilt and passed to QEMU.
- Inspect vector-table and section addresses.
- Check that the machine name matches the linker script's memory model.
- Confirm startup reaches `main()`.
- Check the output configuration (semihosting options, call to `semihost_write0`, console routing).

Symptom	Where to look
Compiler rejects a command	Cross-toolchain installation and Makefile flags
Unresolved symbol	Object list and matching declarations
Section overflow	Linker-script memory regions and the map file
QEMU starts but prints nothing	Reset path, output call, semihosting options

- Disciplined build loop: build with `make`, read the command and the **first** diagnostic, run the ELF in QEMU, compare observed and expected output.
- Compilation failures, boot failures, and program-logic bugs must be kept separate.

> **TIP**
>
> Locate the failing boundary Changing random compiler flags is not a diagnosis; the first step is to identify which boundary of the pipeline failed.

7 Build Automation with Make

7.1 Rules, Prerequisites, and Timestamp-Based Rebuilds

`make` is not a compiler: it reads a `Makefile` describing a graph of **targets** and runs only the commands needed to bring the requested targets up to date. The alternative, typing every compiler command by hand, is slow and error-prone; `make` is the reproducible build.

```
target: prerequisite1 prerequisite2
          recipe line 1
          recipe line 2
```

- `target` : normally a file to produce (object file, ELF image).
- `prerequisites` : files that must be current before the recipe runs.
- `recipe` : shell commands that produce the target; **every recipe line starts with a literal Tab**, not spaces.
- A target is **out of date** and its recipe runs if the target does not exist, or if any prerequisite is newer (by modification time) than the target.
- Targets may depend on other targets, forming a **dependency graph** walked bottom-up:

```
main.c changed      --> main.o rebuilt --> scratch-os.axf relinked
startup.c unchanged --> startup.o reused
```

- The rebuild logic is correct only if the graph is complete:
  - Header dependencies must be tracked so that an interface change recompiles the affected sources.
  - A linker script that is not listed as a prerequisite does not trigger a relink when it changes; `make clean && make` forces a full rebuild.

7.2 Diagnosing the Missing Separator Error

```
CC := arm-none-eabi-gcc
BUILD_DIR := build

$(BUILD_DIR)/%.o: %.c
    @mkdir -p $(dir $@)
         $(CC) $(CFLAGS) -c $< -o $@
```

```
$ make
Makefile:5: *** missing separator.  Stop.
```

- `Makefile:5` is exact: line 5 is indented with four spaces instead of a Tab.
- `missing separator` describes what the parser expected (the Tab that marks a recipe line), not what is written: without it, `make` tried to parse the line as an assignment or a rule and failed.
- Spaces and a Tab look identical in an editor; the defect is visible only by rendering the bytes:

```
$ cat -et Makefile
CC := arm-none-eabi-gcc$
BUILD_DIR := build$
$
$(BUILD_DIR)/%.o: %.c$
    @mkdir -p $(dir $@)$
^I$(CC) $(CFLAGS) -c $< -o $@$
```

- `cat -et` marks each line end with `$` and each Tab with `^I`; line 5 lacks the `^I`.
- Portability:
  - `cat -A` exists only in GNU `cat`; BSD `cat` rejects it. `cat -et` works on both.
  - `sed -n l` is a portable alternative (Tab shown as `\t`), but BSD `sed` wraps long lines at about 70 columns.
- General technique: when a tool rejects a line that looks correct, render its bytes instead of rereading its characters.
- Editors can be configured not to expand Tabs in Makefiles.

7.3 Variables and Assignment Flavours

```
CC := arm-none-eabi-gcc
CFLAGS := -mthumb -mcpu=cortex-m3
CFLAGS += -Wall -Wextra
```

Operator	Name	Behaviour
`:=`	Simply expanded	Right-hand side evaluated once, immediately (preferred default)
`=`	Recursively expanded	Re-evaluated at every use; can be slow or loop if self-referential
`+=`	Append	Adds to the current value
`?=`	Conditional	Assigns only if not already set; allows overrides such as `make CC=clang` or the environment

- Variables are expanded with `$(NAME)`.
- Convention: uppercase names (`CC`, `CFLAGS`, `LDFLAGS`) are the tool and flags used by recipes; `make`'s built-in rules look for these exact names. Other names (`SOURCES`, `OBJECTS`, `BUILD_DIR`) hold the Makefile's own bookkeeping.

7.4 Automatic Variables, Pattern Rules, and Wildcards

Variable	Meaning
`$@`	Target of the rule

Variable	Meaning
`$<`	First prerequisite
`$^`	All prerequisites, space-separated, duplicates removed
`$*`	Stem matched by `%` in a pattern rule
`$(dir $@)`	Directory part of `$@` (a built-in function, not an automatic variable)

```
$(BUILD_DIR)/%.o: %.c
	@mkdir -p $(dir $@)
	$(CC) $(CFLAGS) -c $< -o $@
```

- `%.o: %.c` is a **pattern rule**: `%` is a stem that must match on both sides. For `build/src/main.o`, the stem is `src/main` and the prerequisite is `src/main.c`. One rule covers every object file.
- `$(dir $@)` (e.g., `build/boot/`) lets `mkdir -p` create the output directory before the compiler writes into it; `$<` is the `.c` file; `$@` is the `.o` being produced.
- `%` is a `make` mechanism, distinct from shell globbing. The glob equivalent is the `$(wildcard ...)` function, evaluated when the Makefile is read:

```
SOURCES := $(wildcard boot/*.c src/*.c)
```

Source list style	Advantage	Risk
Explicit (`SOURCES := boot/startup.c src/main.c`)	Easy to audit; a renamed file fails loudly	Must be maintained by hand
`$(wildcard ...)`	New files picked up automatically	Silently includes or excludes files

- An explicit list suits small trees; `$(wildcard ...)` suits large trees where manual maintenance is the bigger risk.
- `$(SOURCES:%.c=$(BUILD_DIR)/%.o)` is a **substitution reference**, shorthand for `$(patsubst %.c,$(BUILD_DIR)/%.o,$(SOURCES))`. It maps the source list to `build/boot/startup.o build/src/main.o`, connecting it to the pattern rule.

7.5 Phony Targets and the Default Goal

```
.PHONY: all clean run

all: $(BIN)

run: $(BIN)
	qemu-system-arm -machine mps2-an385 ...

clean:
```

```
	rm -rf $(BUILD_DIR)
```

- `run` and `clean` name actions, not files. If a file named `clean` existed, `make` would consider the target up to date and skip the recipe. `.PHONY` declares that a target is not a file, so its recipe always runs.
- `all` is the conventional **default goal**: the first target in the file, built when `make` is invoked without arguments. With `$(BIN)` as its only prerequisite it is an alias for the real image.

7.6 Inspecting Makefile State

A Makefile can be wrong without failing (e.g., a wrong substitution silently builds the wrong list). The useful information is what variables *expanded to*, not what the text says.

```
SOURCES := boot/startup.c src/main.c
BUILD_DIR := build
OBJECTS := $(SOURCES:%.c=$(BUILD_DIR)/%.o)
CFLAGS := -mthumb -mcpu=cortex-m3
CFLAGS += -Wall -Wextra

$(info OBJECTS = $(OBJECTS))
$(warning CFLAGS is $(CFLAGS))

print-%:
	@echo "$* = $($*)"

all:
	@echo linking
```

```
$ make all
OBJECTS = build/boot/startup.o build/src/main.o
Makefile:8: CFLAGS is -mthumb -mcpu=cortex-m3 -Wall -Wextra
linking
```

- `$(info ...)` and `$(warning ...)` are expanded while the Makefile is **read**, before any recipe runs: they show the value at that point in the file (after the `+=` here).
- `$(warning ...)` adds the `file:line:` prefix, useful when several are present.
- Both fire on every invocation, including `make clean`: they are temporary debugging aids.
- `print-%` uses the pattern mechanism and `$*` to query any variable: `make print-CFLAGS` prints `CFLAGS = -mthumb -mcpu=cortex-m3 -Wall -Wextra`.
- `make -n` prints the commands a goal would run without executing them, to verify that `$<` and `$@` expand as intended.
- `make -p` dumps the whole database (over 1300 lines for a 14-line Makefile, mostly built-in defaults); useful only when the cause lies in something not written in the Makefile.

7.7 The Complete Build Graph

```
SOURCES := boot/startup.c src/main.c
OBJECTS := $(SOURCES:%.c=$(BUILD_DIR)/%.o)

.PHONY: all clean run
all: $(BIN)

$(BUILD_DIR)/%.o: %.c
        @mkdir -p $(dir $@)
        $(CC) $(CFLAGS) -c $< -o $@

$(BIN): $(OBJECTS)
        $(CC) $(CFLAGS) $(LDFLAGS) $(OBJECTS) -o $(BIN)

run: $(BIN)
        qemu-system-arm ...

clean:
        rm -rf $(BUILD_DIR)
```

- Three-level graph: `run` depends on `$(BIN)`, `$(BIN)` on `$(OBJECTS)`, each object on its `.c` file.
- `make` (equivalent to `make all`): builds missing or stale objects through the pattern rule, then runs the link recipe. Running it again with no changes does nothing.
- `make run`: rebuilds anything stale, then boots the image in QEMU.
- The graph is the record of what the image is made of; re-running it is correct whether nothing changed, one file changed, or `build/` does not exist.
- A separate **build directory** holds generated objects, dependency files, maps, and ELF images:
  - Source directories stay readable.
  - `clean` can remove all generated files safely.
  - Different configurations can coexist.
  - A fresh build tests whether the dependency graph is complete.
- Generated objects and ELF bytes are never edited by hand to change program behaviour.

1 Memory Technologies on a Microcontroller

A microcontroller contains two fundamentally different kinds of on-chip memory, and every part of a C program can live in only one of them. The choice is dictated by three properties:

1. Whether the memory sits on the CPU's memory bus (so every byte or word has an address),
2. whether it can be written with an ordinary store instruction
3. whether it keeps its content at power-off.

1.1 SRAM, NOR Flash, and Block Devices

SRAM (static RAM) is **connected directly to the memory bus**:

1. every byte has an address and is read or written with a normal load or store
2. **Fast**: one access takes about one clock cycle
3. **Byte-writable** at any time,
4. **volatile**: at power-off its content is lost, and at power-on it is **undefined** (random bits, not zeros). SRAM is therefore the only place where anything that changes at run time can live: variables and the stack.

NOR flash is also on the memory bus, like a ROM:

1. Every word has an address and random-access reads are fast.
2. The CPU can therefore fetch instructions directly from it, which is called **execute-in-place (XIP)**: no copy is needed before running.
3. **Non-volatile**.
4. **Writing is not a store instruction**: a whole sector (a few KB) must first be **erased**, setting all its bits to 1, and then individual words are **programmed**, which can only turn bits from 1 to 0. Both operations take microseconds to milliseconds and go through a command sequence sent to the flash controller, so at run time NOR flash is effectively **read-only**.

NAND flash (SD cards, SSDs) and hard disks are **not** on the memory bus:

1. Accessed through a controller one block (page) at a time, typically 512 B to 4 KB, and a block has no CPU address.
2. The CPU can neither fetch instructions from it nor read a single variable: content must first be copied into RAM. This is why a desktop OS needs a **loader** that copies the executable from the SSD into RAM. A microcontroller has no loader: its program already sits in NOR flash, at the address it runs from.

Technology	On the bus?	Execute from it?	Write at run time?	Keeps data at power-off?
SRAM	yes	yes	yes, bytes	no (volatile)
NOR flash	yes	yes (XIP)	no (erase/program)	yes
NAND / disk	no (blocks)	no	through a driver	yes

1.2 Placement Rules for a C Program

These properties fix where each part of a C program goes:

Program element	Memory	Reason
Code	Flash	executed in place (XIP)
Constants (`const`, string literals)	Flash	never modified: read in place
Variables	RAM	modified at run time
Initialized variables (`int x = 5;`)	RAM at run time + Flash in the image	RAM is random at power-on: the initial value is stored in flash and copied into RAM by the startup code before `main()` (see 2.2.1)

1.3 Cortex-M Address Space

The CPU sees a single 4 GB address space (32-bit addresses). On every Cortex-M the **purpose** of each region is fixed by the ARM architecture; each chip vendor decides how much memory or which devices populate it.

Address range	Region	On `mps2-an385`
`0x0000_0000` – `0x1FFF_FFFF`	Code	4 MB at `0x0`: the program ("FLASH")
`0x2000_0000` – `0x3FFF_FFFF`	SRAM	4 MB at `0x2000_0000`: "RAM"
`0x4000_0000` – `0x5FFF_FFFF`	Peripherals	UART, timers, …
`0x6000_0000` – `0xDFFF_FFFF`	External RAM / devices	not used
`0xE000_0000` – `0xE00F_FFFF`	Private peripheral bus	registers of the CPU core itself (System Control Space at `0xE000_E000`)

The same map, with the populated parts drawn inside the architectural regions:

Cortex-M3 address space on mps2-an385



Peripherals are controlled by reading and writing their addresses (memory-mapped I/O), and the last region holds registers that configure the CPU core itself.

> **INFO**
>
> Emulated flash On `mps2-an385` (and in QEMU) the 4 MB at address `0x0` is fast RAM preloaded with the program image. It plays the role of flash and is treated as read-only, exactly as real flash on a microcontroller.

2 Linker Script and Image Layout

A **linker script** is a text file, passed to the linker with `-T`, that describes the memory layout of the target chip and how the program must be placed in it.

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

ENTRY(Reset_Handler)

_estack = ORIGIN(RAM) + LENGTH(RAM);
_Min_Stack_Size = 0x1000;
_Min_Heap_Size = 0x1000;

SECTIONS
{
    .isr_vector :
    {
        . = ALIGN(4);
        KEEP(*(.isr_vector))
        . = ALIGN(4);
    } > FLASH

    .text :
    {
        . = ALIGN(4);
        *(.text*)
        *(.rodata*)
        . = ALIGN(4);
        _etext = .;
    } > FLASH

    _sidata = LOADADDR(.data);

    .data :
    {
        . = ALIGN(4);
        _sdata = .;
        *(.data*)
        . = ALIGN(4);
```

```
        _edata = .;
    } > RAM AT > FLASH

    .bss :
    {
        . = ALIGN(4);
        _sbss = .;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _ebss = .;
    } > RAM

    .heap :
    {
        . = ALIGN(8);
        PROVIDE( end = . );
        PROVIDE( _end = . );
        _heap_bottom = .;
        . = . + _Min_Heap_Size;
        _heap_top = .;
    } > RAM

    __StackLimit = _estack - _Min_Stack_Size;
    ASSERT(__StackLimit >= _heap_top, "region RAM overflowed: heap collides with the stack")
}
```

The linker alone only merges the sections of the object files and resolves symbols: it **does not know which memories the chip has** or where they are. The linker script supplies this information. Based on it, the linker:

- assigns a final address to every section and symbol;
- places each section in the right memory (code and constants in flash, variables in RAM);
- defines symbols that the startup code uses, such as section boundaries and the initial stack pointer;
- checks that everything fits in the available memory, failing the link otherwise.

On bare metal there is **no OS loader** to decide where the program goes. The image must already match the hardware: for example, the vector table must be at address `0x0`, and initialized data must be stored in flash and used in RAM. The linker script is what guarantees this.

Internally, a linker script is divided into four parts:

Part	Role
`MEMORY { ... }`	declares the available memory regions: name, start address (`ORIGIN`), size (`LENGTH`), attributes (`r` , `w` , `x`)

Part	Role
`SECTIONS { ... }`	builds the output sections from the input sections of the object files and assigns each one to a memory region
`ENTRY(...)`	names the symbol where execution starts; used by tools (debugger, QEMU), not by the CPU
Symbol assignments (`_estack = ...;`)	create named addresses that the code can use, such as the top of the stack or the bounds of `.data` and `.bss`

2.1 MEMORY: Regions and Size Checking

```
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

_estack = ORIGIN(RAM) + LENGTH(RAM);   /* = 0x20400000 */
```

Each line of `MEMORY` defines one memory region; the two regions correspond to the two populated rows of the memory map (1.3).

Field	Meaning	Example
Name	freely chosen identifier, not a keyword	`FLASH` , `RAM`
Attributes	allowed access: `r` read, `w` write, `x` execute	`(rx)` , `(rwx)`
`ORIGIN`	start address of the region	`0x00000000` , `0x20000000`
`LENGTH`	size of the region	`4M`

The linker uses `LENGTH` to verify that the program fits:

- if the content assigned to a region does not fit, the link fails (e.g. `region 'FLASH' overflowed by 28 bytes`) instead of producing an image that cannot work;
- flash must hold `text + data` as reported by `arm-none-eabi-size` : code and constants, plus the stored initial values of `.data` ;
- `.bss` needs no flash, since it has no stored content;
- example: with `text = 96` and `data = 32` , the minimum FLASH `LENGTH` is exactly 128.

`_estack`. Defined as `ORIGIN(RAM) + LENGTH(RAM)` = `0x20400000` , one byte past the last RAM byte. The stack starts there because on Cortex-M it grows **downwards**: starting at the top of RAM gives it the most room.

2.2 SECTIONS: Output Sections Built from Input Sections

Each object file (`.o`) produced by the compiler or assembler contains its own sections, called **input sections**: `.text` (code), `.rodata` (constants), `.data` , `.bss` . The linker merges them into the sections of the final ELF

file, called **output sections**. The `SECTIONS` block contains one definition per output section, always with the same format:

```
<output_section> :
{
    <rule>            /* which input sections to take, e.g. *(.text*) */
    <rule>            /* rules are applied in order                   */
    <symbol> = .;     /* optional: give a name to the current address */
} > <REGION>          /* memory region where the section is placed    */
```

The output sections defined in this linker script are:

Output section	Content	Region
`.isr_vector`	vector table: initial SP and handler addresses	FLASH, at `0x0`
`.text`	code and read-only data (`.rodata`: constants, string literals)	FLASH
`.data`	initialized global variables	RAM (initial values stored in FLASH)
`.bss`	global variables initialized to zero or not initialized	RAM (zeroed at boot, nothing stored)
`.heap`	space reserved for dynamic allocation (`malloc`)	RAM (only reserved, empty)

The first two, both placed in FLASH, are:

```
.isr_vector :
{
    . = ALIGN(4);
    KEEP(*(.isr_vector))
} > FLASH

.text :
{
    *(.text*)            /* code from every .o          */
    *(.rodata*)          /* constants, string literals  */
    _etext = .;
} > FLASH
```

Element	Meaning
`.text :`	name of the output section created in the final ELF
`*(.text*)`	from every input file (`*`), take every section whose name starts with `.text`
`*(.rodata*)`	same, for the read-only data of every input file
`KEEP(...)`	always keep these input sections, even with garbage collection enabled

Element	Meaning
`.`	**location counter**: the address where the linker is currently placing content
`. = ALIGN(4)`	round the location counter up to a multiple of 4
`_etext = .;`	define a symbol equal to the current address (here, the end of the code)
`> FLASH`	place the output section in the FLASH region

Three consequences follow from these rules:

- **Order of rules = order in memory**: inside `.text`, all the code comes first, then all the read-only data.
- `.rodata` **has no output section of its own**: it is folded into `.text`.
- `.isr_vector` **lands at** `0x0`: it is the first output section placed in FLASH, and `.text` follows right after it.

2.2.1 Run Address (VMA) and Load Address (LMA)

```
_sidata = LOADADDR(.data);
.data :
{
    _sdata = .;       /* start of .data in RAM */
    *(.data*)
    _edata = .;       /* end of .data in RAM   */
} > RAM AT > FLASH
```

Every output section has two addresses:

Address	Meaning	In the script
VMA (virtual memory address, *run address*)	where the program uses the section at run time	`> REGION`
LMA (load memory address)	where the section's bytes are stored in the image	`AT > REGION` (if omitted, LMA = VMA)

For most sections the two coincide; `.data` is the exception:

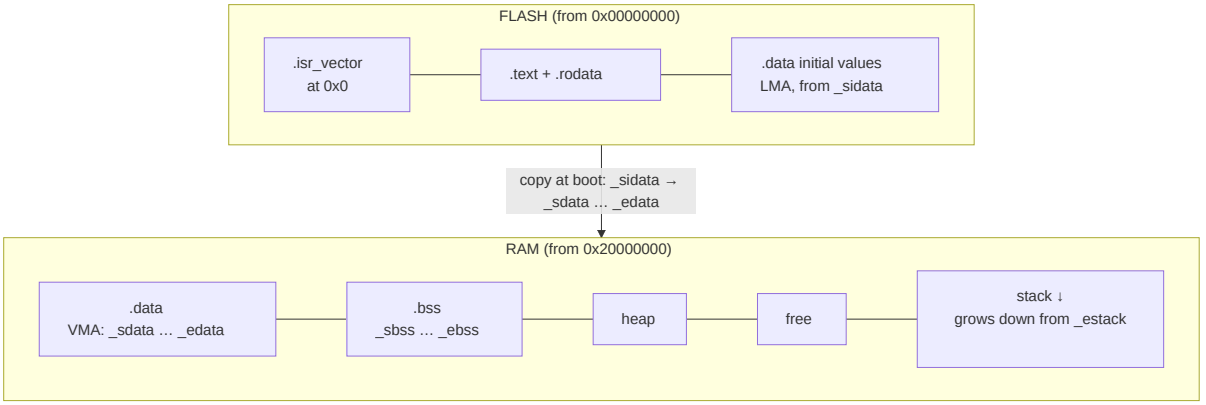
Section	VMA	LMA	Why
`.text`	FLASH	FLASH	code runs where it is stored (XIP)
`.data`	RAM	FLASH	variables are modified in RAM, but their initial values must survive power-off in flash

`> RAM AT > FLASH` means: VMA in RAM, LMA in FLASH. The script also records three addresses for the boot code:

Symbol	Value	Role in the copy
`_sidata`	`LOADADDR(.data)`: start of the initial values in FLASH (LMA)	source
`_sdata`	start of `.data` in RAM (VMA)	destination start
`_edata`	end of `.data` in RAM (VMA)	destination end

> **IMPORTANT**
>
> Nothing copies LMA to VMA automatically The linker only records the two addresses, and the CPU copies nothing at reset. The startup code must copy `.data` from `_sidata` to `_sdata` … `_edata`.

The resulting layout is:



2.2.2 .bss, Heap, and Stack Reservation

These three RAM areas occupy **no space in flash**: the image records only their size or bounds, never their bytes.

- `.bss`: storing zeros in flash would be wasteful, so only the size is recorded.
- `.heap`: it has no initial content at all; it is just space.
- **stack**: it is not even a section, only an address range below `_estack`.

```
.bss :
{
    _sbss = .;
    *(.bss*)
    *(COMMON)
    _ebss = .;
} > RAM

.heap :
{
    . = ALIGN(8);
    _heap_bottom = .;
    . = . + _Min_Heap_Size;
    _heap_top = .;
} > RAM

__StackLimit = _estack - _Min_Stack_Size;
ASSERT(__StackLimit >= _heap_top, "region RAM overflowed: heap collides with the stack")
```

Area	What it is	Bounds	Who prepares it
`.bss`	variables that start at zero	`_sbss` … `_ebss`	boot code fills it with zeros
`.heap`	space reserved for dynamic allocation (`malloc`)	`_heap_bottom` … `_heap_top` (4 KB)	nobody: it is only reserved
stack	**not a section**: the top 4 KB of RAM	`__StackLimit` … `_estack`	SP is set to `_estack` at boot

Element	Meaning
`*(COMMON)`	globals declared without a value (`int counter;`) that some compilers put in a "common" pool: they belong in `.bss` too
`. = . + _Min_Heap_Size`	advance the location counter by 4 KB, so nothing else can be placed there
`__StackLimit`	`_estack - _Min_Stack_Size` : the lowest address the stack may reach
`ASSERT(cond, msg)`	make the link fail with `msg` if `cond` is false

IMPORTANT

Heap–stack collision caught at link time The `ASSERT` makes the **link fail** if the heap would run into the stack limit. A memory corruption that would appear at run time, and be very hard to debug, becomes a clear error at build time.

EXAMPLE

RAM `LENGTH = 8K` `_estack` moves to `0x20002000` and `__StackLimit` to `0x20001000` . The sections `.data` + `.bss` + `.heap` still fit inside 8 KB, so there is no region overflow. However, the 4 KB heap now crosses `__StackLimit` , and the link fails because of the `ASSERT` .

2.2.3 Linker-Defined Symbols

Symbols such as `_sdata` , `_edata` , `_sidata` , `_sbss` , `_ebss` and `_estack` have **no storage**: no memory cell holds their value. The linker only assigns each one an **address**, and the address itself is the information.

```
extern uint32_t _sdata, _edata;   /* "there is something at this address" */
uint32_t *start = &_sdata;        /* correct: the address (0x20000000)    */
uint32_t  oops  = _sdata;         /* wrong: reads RAM at that address     */
```

They are declared as `extern` objects only so that C allows taking their address with `&` . Reading `_sdata` itself would read whatever happens to be in RAM at `0x20000000` . In assembly, `ldr r0, =_sdata` means `r0 = &_sdata;` . The boot code uses these addresses as loop bounds.

2.2.4 Orphan Sections

GCC lets the programmer choose the input section of a variable or function explicitly:

```
int tagged __attribute__((section(".mysection"))) = 42;
```

If no rule in `SECTIONS` matches that section, it is an **orphan**. The linker still places it, guessing from its flags:

- writable and initialized → RAM, right after `.data`, with its own LMA in FLASH;
- read-only → FLASH.

> **WARNING**
>
> Orphans link silently but escape the script's symbols The link succeeds without warnings, but the orphan lies outside every range defined by the script. Here `tagged` lands exactly at `_edata`, so the copy loop over `_sdata` … `_edata` never initializes it: it reads 0 instead of 42.

2.3 Section Placement of C Declarations

Where a C declaration ends up in memory is decided in two steps:

1. the **compiler** chooses the *input section*, based on the kind of declaration;
2. the **linker script** decides which *memory region* that section goes to.

For declarations at file scope (global):

Global declaration	Section	Region
a function body	`.text`	FLASH
`const int k = 3;`	`.rodata`	FLASH
`"a string literal"`	`.rodata`	FLASH
`int x = 5;`	`.data`	RAM, initial value in FLASH
`int y;` or `int z = 0;`	`.bss`	RAM, zeroed at boot

2.3.1 Special Cases

Case	Where it goes	Why
`int z = 0;`	`.bss`	"starts at zero" is exactly what `.bss` guarantees: no reason to store the zero in flash
`const char *p = "hi";`	`"hi"` → `.rodata` (FLASH), `p` → `.data` (RAM)	two distinct objects: the characters are constant, the pointer is an initialized writable variable
local variable (`int local = 7;` inside a function)	**no section**: the stack	it exists only while the function runs and has no fixed address
`static` local (`static int s = 3;` inside a function)	`.data` (or `.bss`)	it keeps one fixed address for the whole run, like a global; only its name is private

> **WARNING**
>
> `const` pointer vs pointer to `const` In `const char *greeting = "hi";` the `const` applies to the **characters**, not to the pointer. The pointer can still be changed (`greeting = "other";`), so it is an initialized writable variable and goes to `.data` (`D`), while `"hi"` goes to `.rodata` . Only `const char * const greeting = "hi";` makes the pointer itself constant, placing it in `.rodata` (`R`) as well. Reading rule: start from the name and read leftwards: "greeting is a pointer to constant chars".

2.4 Inspecting the Linked Image

Three tools show what the linker actually produced: the map file, `nm` and `objdump` .

2.4.1 Map File

The **map file** (generated with the `-Map=` linker option) lists the *Memory Configuration* (the `MEMORY` regions) and the *Linker script and memory map*: every output section, every input section inside it with its source `.o` , every symbol's address, and the `load address` of `.data` . It also lists *Discarded input sections*.

2.4.2 nm: Listing Symbols

`arm-none-eabi-nm` is the GNU binutils tool that lists the **symbols** of an object file (`.o`) or of the final ELF image (`.axf`). A symbol is a name with an address: functions, global variables, linker-defined symbols such as `_sdata` .

For each symbol it prints:

- the **address**;
- a **letter** giving the kind of section the symbol belongs to;
- the **name**.

```
$ arm-none-eabi-nm -n build/scratch-os.axf
00000044 T limit
20000020 D counter
20000024 D greeting
20000028 B total
```

Letter	Meaning
`T` / `t`	code (`.text`)
`R`	read-only data (`.rodata`)
`D` / `d`	initialized data (`.data`)
`B`	zero-initialized data (`.bss`)
`W`	weak symbol
`A`	absolute symbol, belonging to no section

A lowercase letter means the symbol is local to its file: a `static` local `s` appears as `d s.0` , where the `.N`

`.` suffix distinguishes static locals with the same name in different functions. The `-n` option sorts the symbols by address.

2.4.3 objdump: Section Headers and Raw Bytes

`arm-none-eabi-objdump` shows the content of an ELF file. Two options are used:

Option	Output
`-h`	one line per output section with size, VMA and LMA
`-s -j <section>`	the raw bytes of one section

For an image whose startup file contains only a two-word vector table, `objdump -h` shows `.isr_vector` at `0x0` (8 bytes), `.text` from `0x8`, no `.rodata` line (it is inside `.text`), and `.data` with VMA `0x20000000` and an LMA right after the code in flash, equal to `_sidata`.

Each line of an `-s` dump contains:

- the **address** of its first byte;
- **16 bytes**, in four groups of 4, in memory order;
- the same bytes shown as **text** on the right (`.` for non-printable bytes).

2.5 Worked Example

```
const int limit = 100;
int counter = 5;
int total;
int zero = 0;
const char *greeting = "hi";
char buffer[64];
```

Object	Section	`nm` in `main.o`	`nm` in final image
`limit`	`.rodata`	`R`	`T`
`counter`	`.data`	`D`	`D`
`total`	`.bss`	`B`	`B`
`zero`	`.bss`	`B`	`B`
`greeting`	`.data`	`D`	`D`
`buffer`	`.bss`	`B`	`B`
`"hi"`	`.rodata`	not listed (no name)	not listed

> **INFO**
>
> Why `limit` changes from `R` to `T` `nm` classifies a symbol by the output section it ends up in. The linker script merges every `.rodata` into the `.text` output section, so in the final image `limit` is reported as `T`. The bytes are the same, in the same flash; only the name of the containing section changed.

The bytes of the image confirm the layout:

```
$ arm-none-eabi-objdump -s -j .text build/scratch-os.axf
 0038 00bffde7 4c000000 00000020 64000000  ....L...... d...
 0048 68690000 6d61696e 3a206865 6c6c6f20  hi..main: hello

$ arm-none-eabi-objdump -s -j .data build/scratch-os.axf
 20000020 05000000 48000000                    ....H...
```

Address	Bytes	Meaning
`0x44` (FLASH)	`64 00 00 00`	`limit` = 100 (`0x64`)
`0x48` (FLASH)	`68 69 00`	`"hi"` : `'h'` , `'i'` , terminating `'\0'`
`0x20000020` (RAM)	`05 00 00 00`	`counter` = 5
`0x20000024` (RAM)	`48 00 00 00`	`greeting` = `0x00000048`

> **IMPORTANT**
>
> Little-endian The Cortex-M3 stores the **lowest byte first**. The bytes `05 00 00 00` are therefore the number 5, and `48 00 00 00` the number `0x00000048` .

The last row is the key point: `greeting` holds `0x48` , which is exactly the flash address where `"hi"` starts. **A pointer is simply a variable whose value is an address.**

Variables in `.data` appear in definition order. In this example the first 32 bytes of `.data` (`0x20000000` – `0x2000001F`) hold a string defined earlier in the file, so `counter` and `greeting` follow at `0x20000020` and `0x20000024` .

3 Cortex-M3 Programmer's Model and Exception Model

3.1 Registers and xPSR

The **programmer's model** is the set of **registers visible to code** and to the debugger; all are **32 bits** wide. At this level there are no variables: only registers and memory. A C statement such as `x = x + 1` becomes "load `x` into a register, add 1, store it back".

Register	Name	Role
R0–R12	general purpose	values; a C call passes its first arguments in R0–R3
R13	SP	stack pointer: address of the top of the stack
R14	LR	link register: return address after a function call
R15	PC	program counter: address of the next instruction
—	xPSR	program status: flags, Thumb bit, current exception

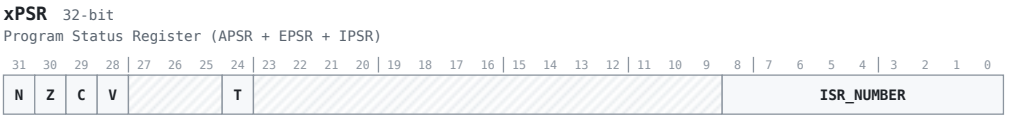
The **stack grows downwards**: a push first does `SP = SP - 4`, then writes at the new SP.

3.1.1 xPSR: the Program Status Register

R0–R15 hold **data or addresses**. xPSR holds neither: it describes the **current state of the processor**. It records the outcome of the last comparison, the instruction set in use, and which exception (if any) is being handled.

The name stands for *Program Status Register*. The `x` indicates that it is the union of three sub-registers, each owning different bits of the same 32-bit word:

Sub-register	Bits	Content	Used for
APSR (Application)	31–28	flags **N Z C V**: negative, zero, carry, overflow	set by `cmp`, read by conditional branches
EPSR (Execution)	24	**T**: Thumb state, always 1 on Cortex-M	instruction set check
IPSR (Interrupt)	8–0	number of the exception being handled (0 = none)	identifying the execution mode



Bits	Field	Description
31	`N`	Negative
30	`Z`	Zero
29	`C`	Carry
28	`V`	Overflow
24	`T`	Thumb state, always 1
8:0	`ISR_NUMBER`	Exception number (0 = Thread mode)

EXAMPLE

Flags in action `cmp r1, r2` computes `r1 - r2`, discards the result and only updates the flags: Z = 1 if the two are equal, C = 1 if `r1 >= r2` (unsigned). The next instruction `bhs done` reads C and jumps if it is 1. This is how every `if` and loop condition works at machine level.

The exception-number field (IPSR) determines the execution **mode**:

Mode	What runs	IPSR (bits 8–0)
Thread mode	normal code: `Reset_Handler`, `main()` and their callees	0
Handler mode	an exception handler	the number of that exception

3.2 Exceptions and Their Priorities

An **exception** is any event that makes the **CPU stop** the current code and run a specific **handler** in Handler mode. An **interrupt** is an exception raised by a hardware device (timer, UART, …).

#	Exception	Raised by
0	(none)	slot holds the initial SP
1	Reset	power-on, reset button
2	NMI	non-maskable interrupt
3	HardFault	any fault with nowhere else to go
4	MemManage	violation of memory-protection rules
5	BusFault	access to an address where nothing answers, or that refuses the access
6	UsageFault	bad instruction use: undefined opcode, Thumb bit 0, divide by 0, …
7–10	reserved	slot must hold 0

#	Exception	Raised by
11	SVCall	the `svc` instruction
12	DebugMonitor	debug events with a debug monitor (unused)
13	reserved	slot must hold 0
14	PendSV	a request raised by software
15	SysTick	the core's built-in timer
16+	IRQs	device interrupts

At reset only **HardFault** is active: MemManage, BusFault and UsageFault are disabled, so every fault **escalates** to HardFault.

When exceptions compete, the more urgent one runs first and can preempt a less urgent handler. On Cortex-M **a lower number means more urgent**. Reset (−3), NMI (−2) and HardFault (−1) have fixed priorities above anything configurable; all other exceptions have a configurable one-byte priority.

> **WARNING**
>
> Lockup If a fault occurs while the CPU is already at HardFault priority (−1), or the fault cannot be reported (e.g. exception entry needs a stack that does not exist), there is nowhere to escalate. The CPU enters **lockup**, reported by QEMU as `Lockup: can't escalate 3 to HardFault (current priority -1)` .

3.3 Reset Sequence and the Vector Table

When the Cortex-M3 leaves reset, **before any software runs**, the hardware performs exactly two reads:

1. the 32-bit word at address `0x0` is loaded into **SP**;
2. the word at address `0x4` is loaded into **PC**, and execution starts there.

There is **no BIOS**, no loader and **no C runtime**. Copying `.data`, zeroing `.bss`, configuring the CPU and calling `main()` are the job of the code at the address stored in the second word.

These two words are the beginning of the **vector table**: a **table of addresses** where slot N holds the address of the handler of exception N. When exception N occurs, the CPU reads slot N and jumps there. Slot 0 is special: it holds the initial SP value, not a code address. The table lives in section `.isr_vector`, which the linker script places first in FLASH, so it lands at address `0x0`, exactly where the CPU reads it at reset.

Cortex-M3 vector table (Scratch OS)

0x0000_003C	[15] **SysTick_Handler** SysTick
0x0000_0038	[14] **PendSV_Handler** PendSV
	⋮ 1 entry (13)
0x0000_0030	[12] **DebugMon_Handler** Debug monitor
0x0000_002C	[11] **SVC_Handler** SVCall
	⋮ 4 entries (7–10)
0x0000_0018	[6] **UsageFault_Handler** UsageFault
0x0000_0014	[5] **BusFault_Handler** BusFault
0x0000_0010	[4] **MemManage_Handler** MemManage
0x0000_000C	[3] **HardFault_Handler** HardFault
0x0000_0008	[2] **NMI_Handler** NMI
0x0000_0004	[1] **Reset_Handler** Reset
0x0000_0000	[0] **_estack** Initial SP (0x20400000)

> **WARNING**
>
> The vector table is in flash, not RAM At reset the table sits at address `0x0`, the start of the **Code/FLASH** region. RAM starts at `0x20000000` and holds only `.data`, `.bss`, heap and stack. The table can be relocated later through VTOR (e.g. into RAM), but this is a deliberate software choice.

The value stored for a handler is its address with **bit 0 set** (e.g. `0x9` for a handler at `0x8`). Instructions are always at even addresses, so bit 0 of a code address carries no information, and ARM reuses it as the **Thumb bit**. The Cortex-M executes only Thumb code and requires this bit to be 1; it checks the bit, copies it into xPSR.T, clears it and jumps to the even address. A table of 16 words (64 bytes) pushes the first code to `0x40`, so word 1 reads `0x41`.

3.4 KEEP and Section Garbage Collection

With the option `-Wl,--gc-sections`, the linker performs **garbage collection of sections**:

1. it starts from the `ENTRY` symbol (`Reset_Handler`);

2. it follows every reference from one section to another (calls, pointers, addresses);
3. it **drops every input section that nothing references**.

This is useful to trim unused code, but it is dangerous for data that **only the hardware reads**. No code references the vector table: the CPU reads it by itself at reset. For the linker it is therefore unused, and it would be discarded.

`KEEP(*(.isr_vector))` marks those input sections as **always kept**, whatever the garbage collector decides.

`--gc-sections`	`KEEP`	Result
off	present or absent	nothing is collected: `KEEP` changes nothing, it only acts as insurance
on	present	`.isr_vector` is kept at `0x0`: the image works
on	absent	`.isr_vector` is discarded: the image does not boot

> **WARNING**
>
> What happens without `KEEP`
>
> 1. `.isr_vector` shrinks to size 0; the map file lists it under *Discarded input sections*.
> 2. `.text` becomes the first section in FLASH, so `Reset_Handler` lands at `0x0`.
> 3. At reset the CPU reads the first two words at `0x0`, which are now **instructions**, and loads them as SP and PC.
> 4. With a meaningless SP and PC, execution faults immediately and the CPU ends in **lockup**.

4 The Startup Code: `boot/startup.s`

The **startup code** (`boot/startup.s`) is the first code the CPU executes after reset, before `main()`. It contains two parts:

- the **vector table**, from which the hardware loads the initial SP and PC;
- `Reset_Handler`, the function that prepares the machine to run C code.

```
// THE COMPLETE STARTUP CODE

/* Assembler settings */
    .syntax unified             /* modern ARM syntax */
    .cpu cortex-m3              /* only M3 instructions */
    .thumb                      /* Thumb code only */

/* 1. Vector table */
    .section .isr_vector,"a",%progbits
    .global isr_vector
    .type isr_vector, %object
isr_vector:
    .word _estack               /* 0: initial SP */
    .word Reset_Handler         /* 1: reset */
    .word NMI_Handler           /* 2: NMI */
    .word HardFault_Handler     /* 3: HardFault */
    .word MemManage_Handler     /* 4: MemManage */
    .word BusFault_Handler      /* 5: BusFault */
    .word UsageFault_Handler    /* 6: UsageFault */
    .word 0                     /* 7: reserved */
    .word 0                     /* 8: reserved */
    .word 0                     /* 9: reserved */
    .word 0                     /* 10: reserved */
    .word SVC_Handler           /* 11: SVCall */
    .word DebugMon_Handler      /* 12: debug monitor */
    .word 0                     /* 13: reserved */
    .word PendSV_Handler        /* 14: PendSV */
    .word SysTick_Handler       /* 15: SysTick */
    .size isr_vector, . - isr_vector

/* Default handlers: weak aliases of Default_Handler */
    .weak      NMI_Handler
    .thumb_set NMI_Handler, Default_Handler
    .weak      HardFault_Handler
    .thumb_set HardFault_Handler, Default_Handler
    .weak      MemManage_Handler
```

```
    .thumb_set MemManage_Handler, Default_Handler
    .weak      BusFault_Handler
    .thumb_set BusFault_Handler, Default_Handler
    .weak      UsageFault_Handler
    .thumb_set UsageFault_Handler, Default_Handler
    .weak      SVC_Handler
    .thumb_set SVC_Handler, Default_Handler
    .weak      DebugMon_Handler
    .thumb_set DebugMon_Handler, Default_Handler
    .weak      PendSV_Handler
    .thumb_set PendSV_Handler, Default_Handler
    .weak      SysTick_Handler
    .thumb_set SysTick_Handler, Default_Handler

/* 2. Reset_Handler */
    .section .text,"ax",%progbits
    .global Reset_Handler
    .type Reset_Handler, %function
    .thumb_func
Reset_Handler:

    /* Step 1: mask interrupts */
    cpsid i                     /* PRIMASK = 1 */

    /* Step 2: set SP */
    ldr r0, =_estack
    mov sp, r0

    /* Step 3: copy .data FLASH -> RAM */
    ldr r0, =_sidata            /* src */
    ldr r1, =_sdata             /* dst */
    ldr r2, =_edata             /* end */
copy_data:
    cmp r1, r2                  /* while (dst < end) */
    bhs copy_data_done
    ldr r3, [r0], #4            /* r3 = *src++ */
    str r3, [r1], #4            /* *dst++ = r3 */
    b copy_data
copy_data_done:

    /* Step 4: zero .bss */
    ldr r1, =_sbss              /* dst */
    ldr r2, =_ebss              /* end */
```

```
    movs r3, #0
zero_bss:
    cmp r1, r2                  /* while (dst < end) */
    bhs zero_bss_done
    str r3, [r1], #4            /* *dst++ = 0 */
    b zero_bss
zero_bss_done:

    /* Step 5: VTOR = &isr_vector */
    ldr r0, =0xE000ED08
    ldr r1, =isr_vector
    str r1, [r0]

    /* Step 6: SHCSR |= bits 16-18 */
    ldr r0, =0xE000ED24
    ldr r1, [r0]
    orr r1, r1, #0x00070000
    str r1, [r0]

    /* Step 7: CCR |= DIV_0_TRP */
    ldr r0, =0xE000ED14
    ldr r1, [r0]
    orr r1, r1, #(1 << 4)
    str r1, [r0]

    /* Step 8: priorities (SHPR1-3) */
    ldr r0, =0xE000ED18
    movs r1, #0x00
    strb r1, [r0, #0]           /* MemManage */
    strb r1, [r0, #1]           /* BusFault */
    strb r1, [r0, #2]           /* UsageFault */
    movs r1, #0x80
    strb r1, [r0, #7]           /* SVCall */
    movs r1, #0xE0
    strb r1, [r0, #10]          /* PendSV */
    strb r1, [r0, #11]          /* SysTick */

    /* Step 9: unmask interrupts */
    cpsie i                     /* PRIMASK = 0 */

    /* Step 10: call main, then hang */
    bl main
hang:
```

```
    b hang

    .ltorg                      /* literal pool */
    .size Reset_Handler, . - Reset_Handler

/* 3. Default_Handler */
    .global Default_Handler
    .type Default_Handler, %function
    .thumb_func
Default_Handler:
    b Default_Handler           /* for (;;) {} */
    .size Default_Handler, . - Default_Handler
```

The order of the steps is dictated by dependencies: nothing may interrupt a half-built system, C needs a stack, globals must hold their values before any C code reads them, and the configuration must be complete before interrupts are accepted.

Step	Action	Reason
1	mask interrupts (`cpsid i`)	nothing may interrupt a half-built system
2	set SP	C functions need a stack
3	copy `.data` FLASH → RAM	initialized globals get their values
4	zero `.bss`	C guarantees uninitialized globals start at 0
5–8	configure the CPU	vector table address, fault handlers, divide-by-zero trap, priorities
9	unmask interrupts (`cpsie i`)	setup is complete
10	`bl main` , then loop forever	hand over to C; on bare metal there is nothing to return to

It works in sequence:

1. **Hardware**: at reset the CPU loads SP from word 0 and PC from word 1 of the vector table, so execution starts at `Reset_Handler` .
2. **Stack**: `Reset_Handler` sets SP to `_estack` , the top of RAM.
3. **`.data`** : it copies the initial values of the initialized variables from FLASH (`_sidata`) to RAM (`_sdata` … `_edata`).
4. **`.bss`** : it fills `_sbss` … `_ebss` with zeros.
5. **CPU configuration**: it sets up the vector table address, fault handlers and priorities.
6. **`main()`** : it calls `main` and, if `main` ever returns, loops forever.

All the addresses used in steps 2–4 are symbols defined by the linker script (chapter 2). The two work at different times: the linker script is used only at **build time** to place sections and define those symbols, and it is never executed; the startup code runs on the CPU at **run time** and uses them as loop bounds. The full code and each step are covered in chapter 5.

> **INFO**
>
> Why assembly When the startup code begins, the stack has not been set up yet and `.data` and `.bss` are not initialized. C code takes all of these for granted, so it cannot be trusted until the startup code has finished. On a desktop this job is done by **crt0**, the C library's startup code; on bare metal it is excluded (`-nostartfiles`) and written by hand.

Writing the startup code needs only a small subset of Thumb instructions, described below.

4.1 Load/Store Architecture

ARM arithmetic **operates only on registers**; memory is accessed only by **load** (`ldr` , memory to register) and **store** (`str` , register to memory) instructions.

Assembly	C equivalent
`ldr r3, [r0]`	`r3 = *r0;`
`str r3, [r1]`	`*r1 = r3;`
`ldr r3, [r0], #4`	`r3 = *r0; r0 += 4;` (i.e. `r3 = *src++;`)
`str r3, [r1], #4`	`*r1 = r3; r1 += 4;` (i.e. `*dst++ = r3;`)
`strb r1, [r0, #7]`	`((uint8_t *)r0)[7] = r1;` (one byte at offset 7)

Square brackets mean "the memory at this address". `[r0], #4` is **post-increment**: the address in `r0` is used first, then 4 is added, so a loop walks through memory one word at a time.

4.2 Constants, Labels, and Branches

Assembly	C equivalent
`ldr r0, =_estack`	`r0 = (uint32_t)&_estack;` (any 32-bit value)
`mov sp, r0`	`sp = r0;` (register to register)
`movs r3, #0`	`r3 = 0;` (small constants only)
`copy_data:`	a label: a name for this address
`b copy_data`	`goto copy_data;`
`bl main`	`main();` (branch with link: return address saved in LR)
`cmp r1, r2`	compute `r1 - r2` , keep only the N Z C V flags
`bhs done`	`if (r1 >= r2) goto done;` (higher or same, unsigned)
`orr r1, r1, #0x10`	`r1 |= 0x10;`
`cpsid i` / `cpsie i`	`__disable_irq()` / `__enable_irq()`

An instruction is only 16 or 32 bits long and cannot embed an arbitrary 32-bit constant. For `ldr rX, =value` the

assembler stores the constant in a nearby table, the **literal pool**, and emits a PC-relative load, which the debugger shows as `ldr r0, [pc, #92]`. The `.ltorg` directive places the pool after the last instruction, where the CPU never executes it.

A C `while` loop becomes "test at the top, jump back at the bottom":

```
loop:   cmp r1, r2          @ while (r1 < r2) {
        bhs done            @
        ...                 @     body
        b   loop            @ }
done:
```

4.3 Assembler Directives

Lines starting with a dot produce no instructions; they steer the assembler.

```
    .section .isr_vector,"a",%progbits   @ following content goes into .isr_vector
    .global isr_vector                   @ name visible to the linker
isr_vector:
    .word _estack                        @ one 32-bit value
    .word Reset_Handler

    .section .text,"ax",%progbits
    .global Reset_Handler
    .thumb_func                          @ next label is Thumb code: bit 0 = 1
Reset_Handler:
```

The equivalent vector table in C is:

```
__attribute__((section(".isr_vector")))
const uint32_t isr_vector[] = { (uint32_t)&_estack, (uint32_t)Reset_Handler };
```

Directive	Effect
`.section name,"flags"`	selects the section; `a` = occupies space in the image, `x` = executable
`.global name`	exports the name to other files and the linker (otherwise it behaves like a `static` C name)
`.thumb_func`	marks the next label as Thumb code, so its address is stored with bit 0 set
`.word X`	stores the 32-bit value X
`.type`, `.size`	tell `nm` and gdb whether a name is code or data, and its size
`.weak name`	declares a weak symbol, overridable by a normal definition
`.thumb_set a, b`	makes `a` an alias of `b`, marked as Thumb code

Directive	Effect
`.ltorg`	emits the literal pool here

5 Reset Handler: From Reset to main()

`Reset_Handler` is the function the CPU executes **first** after every reset. Its address is **stored in slot 1 of the vector table**: at reset the hardware loads that word into PC, so execution starts here, before any C code.

On bare metal there is no operating system and no C library to prepare the machine. `Reset_Handler` takes their place: it brings the CPU and the memory into the state that C code assumes, configures how the CPU reacts to errors, and only then calls `main()`. Its work is organized in two jobs, carried out in order:

Job	Steps	Without it
Make C code safe to run	set the stack, copy `.data`, zero `.bss`	globals hold wrong values, function calls have no valid stack
Configure the CPU	vector table address, separate fault handlers, divide-by-zero trap, priorities	errors pass silently or all end up in the same HardFault

EXAMPLE

A startup that only calls `main` Consider a minimal `Reset_Handler` that executes only `bl main`, and a global `char data_message[] = "hello";`:

- the linker places the bytes `"hello"` in **FLASH** (LMA) and reserves space for `data_message` in **RAM** (VMA);
- nobody copies the bytes from FLASH to RAM;
- `main` reads `data_message` from **RAM**, where the copy was never written.

The result depends on what RAM contains at power-on. QEMU fills RAM with zeros, so the first byte is `'\0'` and the string reads as **empty**. Real hardware leaves random content, so the string reads as **garbage**. The program compiles and runs, but the data is wrong: this is the kind of silent error the copy loop prevents.

5.1 Weak Default Handlers

The vector table has 16 slots, and the CPU jumps to whatever address slot N contains when exception N occurs. Every slot must therefore point to valid code, even for exceptions the program does not handle yet; reserved slots simply hold `0`. Instead of writing nine real handlers, the startup code uses two pieces:

- **`Default_Handler`**: a single catch-all function, an infinite loop (`for (;;) {}`);
- **weak names**: every handler name in the table is declared `.weak` and, by default, is an alias of `Default_Handler` (same address).

A **weak** symbol is a default that gives way: if another file defines a normal (strong) function with the same name, the linker uses that one and drops the weak alias. A handler is installed simply by writing a C function with the right name, without touching the assembly. The C equivalent of each weak name is:

```
void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler")));
```

With only `UsageFault_Handler` defined in `main.c`:

Slot	Name	Defined in C?	CPU jumps to	`nm`
3	`HardFault_Handler`	no	`Default_Handler`	`W`, same address as `Default_Handler`
6	`UsageFault_Handler`	yes	the C function	`T`, its own address
15	`SysTick_Handler`	no	`Default_Handler`	`W`, same address as `Default_Handler`

> **INFO**
>
> Why an infinite loop An unexpected exception is a bug. Freezing at a known place lets a debugger show immediately that PC is in `Default_Handler`, instead of letting the CPU run off into random memory.

5.2 Interrupt Masking and Stack Setup

`cpsid i` sets **PRIMASK**, a 1-bit core register (not memory-mapped): while it is 1, every exception with configurable priority is ignored; only NMI and HardFault get through. Interrupts arriving while masked are not lost: they remain **pending** and run as soon as `cpsie i` clears PRIMASK. PRIMASK is 0 at reset; the boot code sets it first and clears it only after the vector table, handlers and priorities are in place.

Instruction	PRIMASK	C (vendor headers)
`cpsid i`	1: masked	`__disable_irq()`
`cpsie i`	0: accepted	`__enable_irq()`

SP is set explicitly even though the hardware already loaded it from vector word 0, because `Reset_Handler` is not always reached through a real reset: a bootloader or debugger may jump to it with SP holding any value. SP is a special register, so the address is first loaded into `r0` with `ldr` and then moved with `mov sp, r0`.

> **WARNING**
>
> Failure modes of a wrong SP With SP = `0x30000000` (outside RAM), the first push in `main` writes to a non-existent address and faults; reporting the fault requires stacking registers, which faults again, and the CPU enters lockup. With SP = `0x20000001`, the CPU silently clears the two low bits (SP is always word-aligned), so the stack starts at the bottom of RAM and grows *below* it, into an unmapped range where QEMU ignores writes: every call fails silently and nothing is printed.

5.3 Initializing .data and .bss

The `.data` copy translates `while (dst < &_edata) *dst++ = *src++;` with `r0` = `src` (`_sidata`), `r1` = `dst` (`_sdata`) and `r2` = end (`_edata`). It copies whole words, which is safe because the linker script aligns the start and end of `.data` to 4 bytes. Stepping through the loop in a debugger shows `r1` climbing `0x20000000`, `0x20000004`, … until it equals `&_edata`; only then does the string appear in RAM.

The `.bss` loop has the same shape without a source pointer: `while (dst < &_ebss) *dst++ = 0;`. When `.bss` is empty, `_sbss` equals `_ebss` and the loop exits immediately. Relying on the emulator zeroing RAM is incorrect: real SRAM is random at power-on. After these two loops, `Reset_Handler` provides exactly what a C runtime startup (crt0) provides.

5.4 System Control Block Registers

Most core settings are 32-bit registers mapped at fixed addresses in the **System Control Space** (`0xE000E000` – `0xE000EFFF`). The group controlling exceptions is the **System Control Block (SCB)**. They are accessed with ordinary loads and stores:

```
#define SCB_SHCSR (*(volatile uint32_t *)0xE000ED24)
SCB_SHCSR |= (1u << 16);
```

```
    ldr r0, =0xE000ED24   @ address
    ldr r1, [r0]          @ read
    orr r1, r1, #0x10000  @ modify
    str r1, [r0]          @ write
```

This is the **read-modify-write** pattern: read the current value, set only the required bits, write back, so all other bits keep their value. `volatile` forces the compiler to perform every access, in order: it may neither skip a read ("the value cannot have changed") nor remove a write ("nobody reads it"), assumptions that are false for hardware registers. As an example, the CPUID register at `0xE000ED00` reads `0x410fc231` : implementer ARM (`0x41`), part Cortex-M3 (`0xc23`).

Register	Address	Role	Value after boot
VTOR	`0xE000ED08`	address of the vector table	`0x00000000`
CCR	`0xE000ED14`	what counts as a fault	`0x00000210`
SHPR1–3	`0xE000ED18` – `0xE000ED20`	system handler priorities	`0x00000000` `0x80000000` `0xe0e00000`
SHCSR	`0xE000ED24`	enable and status of system handlers	`0x00070000`

5.4.1 VTOR: Vector Table Location

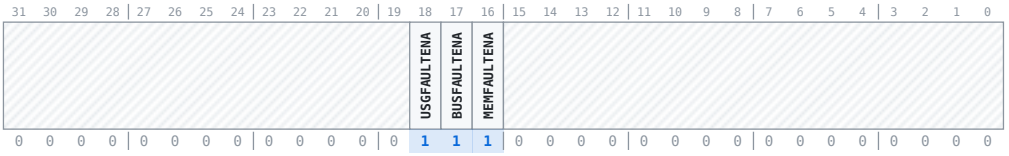
On an exception the CPU reads the handler from **VTOR + 4 × N**, not from `4 × N` . VTOR resets to `0x00000000` , which is why the table must be at address 0 at reset. Bits 31–7 hold the table address, which must be aligned to at least 128 bytes. The whole register is written, so no read-modify-write is needed. Writing it is redundant when the table is already at 0, but it makes the boot code correct wherever the image is placed: a **bootloader** at address 0 may start an application stored higher in flash, which then points VTOR at its own table; or a program may copy its table into RAM to swap handlers at run time.

5.4.2 SHCSR: Separate Fault Handlers

Bit	Name	When set
16	MEMFAULTENA	MemManage faults run `MemManage_Handler`
17	BUSFAULTENA	bus faults run `BusFault_Handler`

Bit	Name	When set
18	USGFAULTENA	usage faults run `UsageFault_Handler`

SHCSR 32-bit = 0x00070000
System Handler Control and State Register (0xE000ED24), value after boot



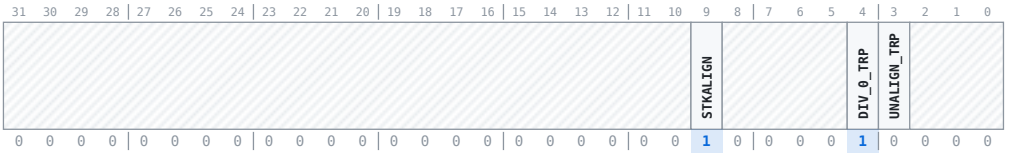
Bits	Field	Value	Description
18	`USGFAULTENA`	1	UsageFault handler enabled
17	`BUSFAULTENA`	1	BusFault handler enabled
16	`MEMFAULTENA`	1	MemManage handler enabled

At reset these bits are 0, so every fault escalates to HardFault and a single handler cannot tell what went wrong. Setting bits 16–18 (`|= 0x00070000`) routes each fault to its own handler. The other SHCSR bits report status (active or pending handlers), so the register must be updated with read-modify-write, never overwritten.

5.4.3 CCR: Divide-by-Zero and Alignment Traps

Bit	Name	Meaning	Setting
3	UNALIGN_TRP	trap unaligned word/halfword accesses	left off
4	DIV_0_TRP	trap integer divide by 0 as a UsageFault	set
9	STKALIGN	8-byte-align the stack on exception entry	already 1 at reset

CCR 32-bit = 0x00000210
Configuration and Control Register (0xE000ED14), value after boot



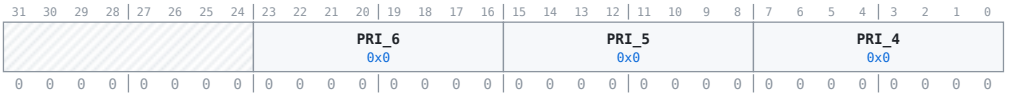
Bits	Field	Value	Description
9	`STKALIGN`	1	8-byte stack alignment on exception entry
4	`DIV_0_TRP`	1	Trap integer divide by zero
3	`UNALIGN_TRP`	0	Trap unaligned accesses

Without DIV_0_TRP, an integer division by zero on Cortex-M3 does not fault: `sdiv` / `udiv` silently return **0**, and the program continues with a wrong value. Setting bit 4 turns this silent bug into a UsageFault. Unaligned accesses are legal (only slower) on Cortex-M3; trapping them would expose sloppy pointer casts but can break correct compiler- or library-generated code, so the trap stays off. Read-modify-write is essential here, since STKALIGN is already 1 and must be preserved: the resulting value `0x210` is bit 4 plus bit 9.

5.4.4 SHPR: System Handler Priorities

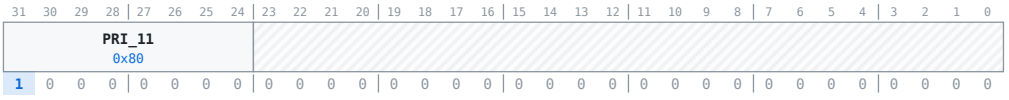
Register	Byte 0	Byte 1	Byte 2	Byte 3
SHPR1 `0xE000ED18`	MemManage	BusFault	UsageFault	—
SHPR2 `0xE000ED1C`	—	—	—	SVCall
SHPR3 `0xE000ED20`	—	—	PendSV	SysTick

SHPR1 32-bit = **0x00000000**
System Handler Priority Register 1 (0xE000ED18)



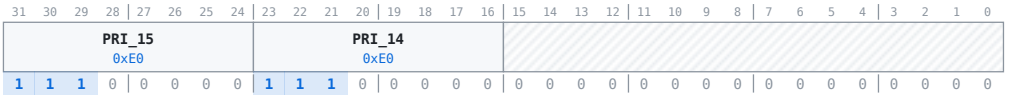
Bits	Field	Value	Description
23:16	`PRI_6`	0x0 (0)	UsageFault priority
15:8	`PRI_5`	0x0 (0)	BusFault priority
7:0	`PRI_4`	0x0 (0)	MemManage priority

SHPR2 32-bit = **0x80000000**
System Handler Priority Register 2 (0xE000ED1C)



Bits	Field	Value	Description
31:24	`PRI_11`	0x80 (128)	SVCall priority

SHPR3 32-bit = **0xE0E00000**
System Handler Priority Register 3 (0xE000ED20)



Bits	Field	Value	Description
31:24	`PRI_15`	0xE0 (224)	SysTick priority
23:16	`PRI_14`	0xE0 (224)	PendSV priority

Each priority is a separate byte, so it is written with a byte store (`strb`) and needs no read-modify-write. Treating SHPR1–3 as a 12-byte array at `0xE000ED18` , the indices are 0–2 (faults), 7 (SVCall) and 10–11 (PendSV, SysTick). The chosen policy is faults most urgent (`0x00`), SVCall intermediate (`0x80`), PendSV and SysTick least urgent (`0xE0`). Real chips implement only the **upper 3 or 4 bits** of each priority byte (lower bits read as 0), so values must differ in their top bits. Reading the words as little-endian, `0x80000000` has `0x80` in byte 3 and `0xe0e00000` has `0xe0` in bytes 2 and 3.

5.5 Fault Handlers in C and the Divide-by-Zero Trap

```c
void UsageFault_Handler( void )
{
    semihost_write0( "UsageFault_Handler: a usage fault happened\n" );
    for ( ;; ) { }
}

int main( void )
{
    volatile uint32_t numerator = 42;
    volatile uint32_t zero = 0;
    volatile uint32_t result = numerator / zero;
    (void)result;
    /* ... */
}
```

`UsageFault_Handler` is a strong definition, so it replaces the weak alias; `HardFault_Handler`, `MemManage_Handler` and `BusFault_Handler` follow the same pattern. Each handler prints its name and loops forever, because a fault is a bug with no sensible way to continue. The divisor is `volatile`: otherwise the compiler would see the constant 0 and compute or remove the division at build time, and no divide instruction would execute. `(void)result;` only suppresses the unused-variable warning.

The outcome isolates the role of each configuration step:

Configuration	Output
SHCSR bits and DIV_0_TRP set	`UsageFault_Handler: a usage fault happened`
SHCSR bits not set	`HardFault_Handler: …` (the fault escalates, its kind is lost)
DIV_0_TRP not set	no fault: the division returns 0 and `main` continues

5.6 Additional Startup Work on Real Chips

QEMU's CPU runs at full speed out of reset. On a real microcontroller the startup code usually does more, often in a vendor function `SystemInit()` called from `Reset_Handler`:

- **Clock and PLL**: the chip starts on a slow internal oscillator, and a PLL multiplies the frequency up to full speed.
- **Flash wait states**: flash is slower than a fast CPU, so the flash controller must be told to insert wait cycles *before* the clock is raised, or the CPU reads garbage.
- **Watchdog**: a timer that resets the chip unless software periodically feeds it; on some chips it runs from reset, so boot code must feed or disable it.
- **FPU**: on a Cortex-M4F the floating-point unit is off at reset and must be enabled in **CPACR** (`0xE000ED88`), otherwise the first float instruction raises a UsageFault.
- **C++**: static constructors listed in `.init_array` must be called before `main`.

5.7 Verifying Boot State with gdb

A **debugger** controls a running program: it stops it at a breakpoint or before the first instruction, steps it one machine instruction or one C line at a time, inspects and modifies registers and memory, and resumes it. On a microcontroller, where a crash often produces no message at all, it is frequently the only way to see what the CPU does. The ARM build of gdb is `arm-none-eabi-gdb` (`gdb-multiarch` on Linux).

Debugging is **remote**: the program runs inside QEMU, and gdb connects over TCP. QEMU is started with the normal run command plus `-S` (freeze the CPU before its first instruction) and `-gdb tcp::1234` (start a gdb server on port 1234). gdb reads the ELF only for symbols and debug information; QEMU already holds the code.

```
qemu-system-arm -machine mps2-an385 -monitor null -semihosting \
    --semihosting-config enable=on,target=native \
    -kernel build/scratch-os.axf -serial stdio -nographic \
    -S -gdb tcp::1234
```

Command	Effect
`target remote :1234`	connect to the gdb server
`info registers sp pc`	show registers (`info registers` shows all)
`x/2wx 0`	examine 2 words in hex at address 0
`x/s &data_message`	examine memory as a C string
`p/x $r1`	print a register or C expression in hex
`display/i $pc`	show the next instruction after every step
`stepi` (`si`)	execute one machine instruction
`break main` (`b`)	stop when execution reaches a symbol or label
`continue` (`c`)	run to the next breakpoint

Attached before the first instruction, gdb already shows SP = `0x20400000` and PC = `Reset_Handler`, while `x/2wx 0` shows `0x20400000 0x00000041` (or `0x9` with a two-word table): no instruction has executed, so the hardware itself loaded both registers from the vector table, clearing the Thumb bit for PC. At `break main`, the SCB registers show the values in the table above and `info registers primask` reads 0 (it reads 0 at reset and 1 after `cpsid i`).

6 Freestanding C and Semihosting

6.1 Hosted and Freestanding Environments

The C standard defines two environments:

Aspect	Hosted	Freestanding
Example	desktop program	bare-metal Cortex-M3 image
Below the program	a C library (libc) and an OS	nothing
`printf`, `malloc`, `fopen`, `exit`	available	not available
Who starts `main()`	libc startup code, invoked by the OS	the application's own startup code

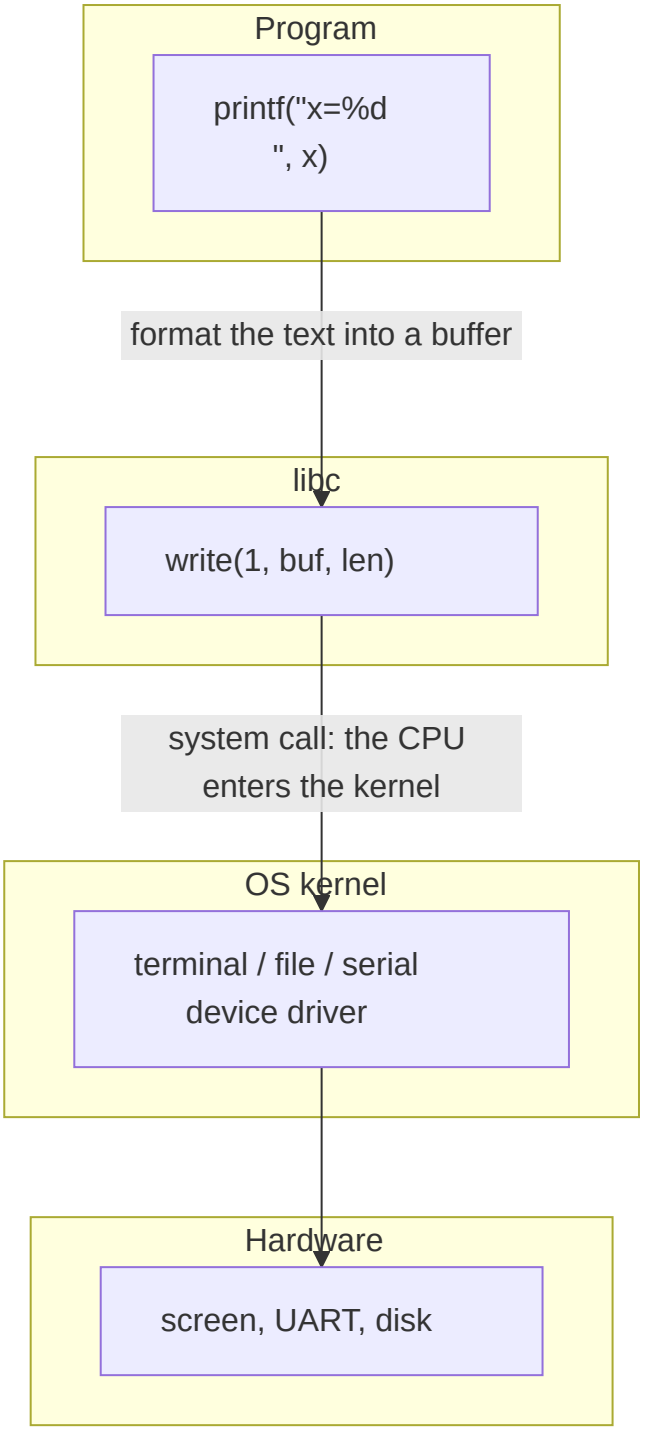
A freestanding environment still provides the headers that need **no run-time code**, only types, constants and macros supplied by the compiler:

Header	Provides
`stdint.h`	`uint32_t`, `int8_t`, …
`stddef.h`	`size_t`, `NULL`, `offsetof`
`stdbool.h`	`bool`, `true`, `false`
`stdarg.h`	`va_list` for variadic functions
`limits.h`, `float.h`	`INT_MAX`, `FLT_EPSILON`, …

There is no `stdio.h`, `stdlib.h` or `string.h`: no printing, no `malloc`, no `exit`, not even `memcpy`. This is why the boot code copies `.data` with its own loop.

6.2 The Layers Under printf

On a desktop, `printf` looks like a **single function** that writes to the screen. In reality it is the top of a stack of layers, and **most of the work happens in layers that a bare-metal system does not have**:



`printf` itself only **formats**: it turns `"x=%d"` and `42` into the characters `x=42` in a buffer. **Getting those characters out of the machine** is the **OS's** job, reached through the `write()` system call. Many other libc functions have the same structure: a portable part on top, and a dependency on something underneath.

libc feature	Requires underneath
`printf`, `puts`	`write()`: a way to send bytes to a device
`malloc`, `free`	a heap region, grown with `sbrk()`
`exit`	something to return to

libc feature	Requires underneath
`main()` being called	crt0: sets up stack, `.data`, `.bss`, then calls `main`

6.2.1 libc on Bare Metal

Embedded C libraries such as **newlib** keep the portable part (formatting, string handling, data structures) and leave the bottom functions (`write`, `sbrk`, `exit`, …) as **hooks**: empty stubs that must be implemented for each board.

This course links **no libc at all**, for three reasons:

Reason	Explanation
Nothing to put under the hooks	there is no kernel, no `write()`, no heap manager yet
crt0 is not needed	`Reset_Handler` already does its work
No hidden code	the image contains only the startup file, the application and a few compiler helpers

> **INFO**
>
> What replaces `printf` Without libc, output is produced by the program itself: in this chapter through **semihosting** (6.4), and from week 3 through a UART driver and the course's own `kprintf` / `printf`.

6.3 Build Flags for a Freestanding Image

```
CFLAGS := -mthumb -mcpu=cortex-m3 -nostartfiles -ffreestanding -std=gnu17
LDFLAGS += -nostdlib -lgcc
```

Flag	Meaning
`-ffreestanding`	GCC must not assume a libc or OS exists (e.g. that a function named `printf` behaves like the standard one)
`-nostartfiles`	do not link the toolchain's startup code (crt0)
`-nostdlib`	do not link any standard library or startup file (no `-lc`)
`-lgcc`	add back libgcc, the compiler's own helper routines

`libgcc` is kept because the compiler sometimes turns a plain C operator into a **call** to a helper when the CPU has no single instruction for it. These helpers need neither OS nor libc, and dropping them would make harmless code fail to link.

C code	CPU	GCC emits
`a / b` (32-bit)	Cortex-M0 (no divide instruction)	`bl __aeabi_idiv`
`a / b` (32-bit)	Cortex-M3	`sdiv` (hardware)

C code	CPU	GCC emits
`x / y` (64-bit)	Cortex-M3	`bl __aeabi_uldivmod`

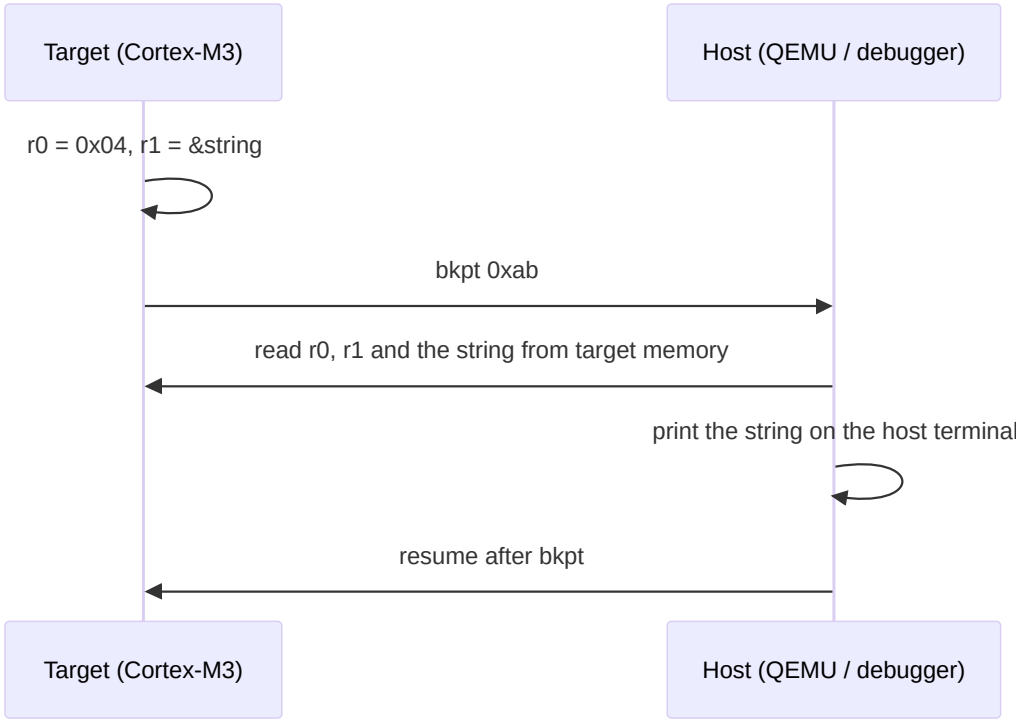
Calling `printf` in such a build always fails: with a toolchain that ships no C library, compilation stops with `stdio.h: No such file or directory`; with a toolchain that ships newlib, the header is found but the link fails with `undefined reference to 'printf'` because of `-nostdlib`.

6.4 Semihosting Output via bkpt 0xab

Without libc and without a UART driver, the program has no way of its own to print anything. **Semihosting** solves this by asking the **host** (the PC running QEMU or the debugger) to perform the I/O on the target's behalf. It is the mechanism that produces every message printed so far.

The request follows a fixed convention:

Register / instruction	Content
`r0`	operation code: `0x04` = `SYS_WRITE0`, print a NUL-terminated string
`r1`	argument: the address of the string
`bkpt 0xab`	a breakpoint with the special immediate `0xab`: "semihosting request"



In C, the request is wrapped in a small function:

```
static void semihost_write0( const char *message )
{
    register uint32_t    r0 __asm__( "r0" ) = 0x04;    /* SYS_WRITE0 */
    register const char *r1 __asm__( "r1" ) = message; /* string address */
```

```
    __asm__ volatile ( "bkpt 0xab" : : "r"( r0 ), "r"( r1 ) : "memory" );
}
```

Element	Meaning
`register … __asm__( "r0" )`	GCC extension that forces a variable into a specific register; standard C gives no control over registers, but the convention requires `r0` and `r1`
`__asm__ volatile ( "bkpt 0xab" … )`	inserts an instruction that no C statement can express; `volatile` prevents the compiler from removing or moving it
`: "r"( r0 ), "r"( r1 )`	the instruction uses these two registers as inputs
`"memory"` clobber	the instruction may read memory (the string), so every pending store must complete before it

WARNING

Only with a host listening Semihosting works only when an emulator or debugger intercepts the `bkpt` (`-semihosting` in QEMU). On a standalone board with no debugger attached, `bkpt` simply halts the CPU.

WARNING

Multiple instructions in one `__asm__` C concatenates adjacent string literals, so `"ldr r2, [r0]" "ldr r3, [r1]"` reaches the assembler as the single line `ldr r2, [r0]ldr r3, [r1]` and fails with `garbage following instruction` . Each instruction must end with `\n` (commonly `\n\t`) inside the string.

INFO

Semihosting is a trap The program stops its own work, control passes to a more powerful entity outside the CPU, which performs the job and then returns control. This is exactly the shape of a **system call** (chapter 7), with the host playing the role of the kernel.

7 Privileged Execution and the SVC System Call

7.1 Privileged and Unprivileged Thread Mode

The Cortex-M3 implements in hardware the kernel-mode/user-mode separation of operating systems, combined with the two execution modes:

CPU mode	Privilege	Typical code
Thread	privileged	`Reset_Handler`, `main` after reset (the kernel)
Thread	unprivileged	application code after `CONTROL.nPRIV = 1`
Handler	privileged	every exception handler

> **IMPORTANT**
>
> Only three combinations nPRIV affects **Thread mode only**: Handler mode is always privileged. An exception therefore always runs privileged, and returns to Thread mode with the privilege it had before.

After reset the CPU is in privileged Thread mode, which is why `Reset_Handler` can write VTOR, SHCSR and CCR. Unprivileged Thread mode is the equivalent of user mode: the mode for code a kernel would not trust.

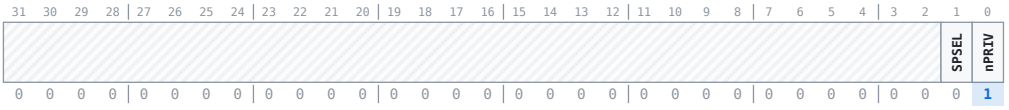
7.2 The CONTROL Register

Thread-mode privilege is selected by **CONTROL**, a special core register (not memory-mapped, like PRIMASK):

Bit	Name	Meaning
0	nPRIV	0 = privileged, 1 = unprivileged (Thread mode only)
1	SPSEL	selects which stack pointer Thread mode uses

CONTROL 32-bit = 0x00000001

CONTROL register after the privilege drop



Bits	Field	Value	Description
1	`SPSEL`	0 → MSP	Thread-mode stack pointer 0 = MSP, 1 = PSP
0	`nPRIV`	1 → unprivileged	Thread-mode privilege 0 = privileged, 1 = unprivileged

Special registers are not reachable with `ldr` / `str`; they are read with `mrs r0, control` (`r0 = CONTROL;`) and written with `msr control, r0` (`CONTROL = r0;`).

7.3 Dropping Privilege

```
    mrs r0, control     @ r0 = CONTROL
    orr r0, r0, #1      @ set nPRIV
    msr control, r0     @ now unprivileged
    isb                 @ discard instructions fetched under the old rules
```

This is a read-modify-write on a special register (`CONTROL = CONTROL | 1;`). The `isb` (instruction synchronization barrier) is required because the CPU prefetches instructions: some may already have been fetched under the privileged rules, and `isb` flushes and refetches them under the new ones.

> **IMPORTANT**
>
> A one-way street Writing CONTROL is itself privileged: an unprivileged `msr control, r0` that clears nPRIV is silently **ignored**. The only ways back to privileged execution are a **reset** or an **exception**, whose handler runs in Handler mode, always privileged.

7.4 Restrictions on Unprivileged Code

Attempt	Result
read/write the System Control Space (`0xE000E000` …): SysTick, NVIC, SCB	BusFault
`msr control, r0` to regain privilege	ignored
`cpsid i` / `cpsie i`	ignored
semihosting `bkpt 0xab`	refused by QEMU by default, so it faults

(The NVIC is the core unit that manages device interrupts, located in the same System Control Space as the SCB.) Ordinary RAM, flash and the program's own code remain accessible: restricted code cannot **reconfigure** the machine or silence it by masking interrupts. A semihosting call after the privilege drop therefore never prints: the refused `bkpt` becomes a fault with no dedicated handler and escalates to HardFault, whose handler *can* print because handlers run privileged.

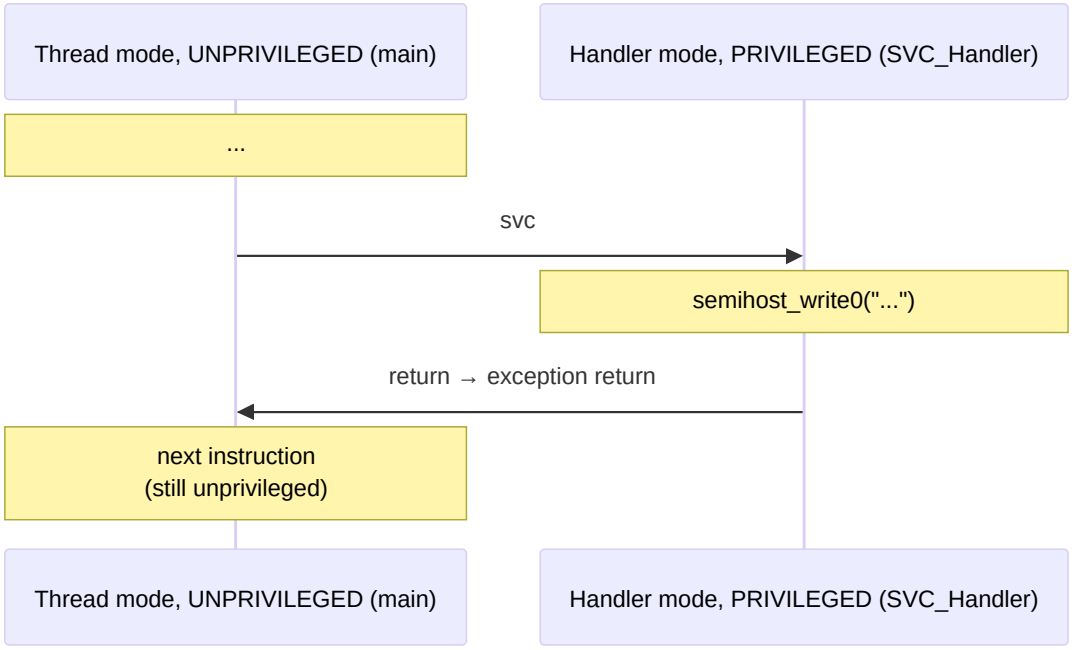
7.5 SVC as a Controlled Entry into Privileged Code

```
void SVC_Handler( void )
{
    semihost_write0( "SVC_Handler: privileged code printing on behalf of unprivileged code\n"
);
}

/* in main(), after the privilege drop */
__asm__ volatile ( "svc #0" : : : "memory" );
```

```
volatile uint32_t *systick_ctrl = ( volatile uint32_t * ) 0xE000E010;
*systick_ctrl = 0;   /* unprivileged store to the System Control Space */
```

`svc #n` (supervisor call) is permitted in unprivileged code. It raises **SVCall** (exception 11), whose handler is in slot 11 of the vector table. Like any exception it runs in Handler mode, hence **privileged**. When the handler returns, execution resumes at the instruction after `svc`, in Thread mode and **still unprivileged**:



Privilege is **borrowed** only for the duration of the handler, and only code placed in the handler runs with it: the caller can request a service but cannot choose what executes. The immediate `#n` is encoded in the instruction; real kernels use such a request number, plus arguments in registers, to select the service.

`SVC_Handler` returns normally instead of looping, because `svc` is a deliberate request, not an error. A plain C function can serve as a handler because on exception entry the Cortex-M automatically saves the registers a C function may clobber (R0–R3, R12, LR, PC, xPSR) and restores them on return. If `SVC_Handler` is not defined, `svc` lands in the weak `Default_Handler` and the program hangs silently.

The final store to SysTick's control register (`0xE000E010`) is refused by the System Control Space and raises a **BusFault**, which reaches `BusFault_Handler` because SHCSR enabled bus faults. Inside `SVC_Handler`, CONTROL still reads `0x1`: the handler is privileged because it runs in Handler mode, not because nPRIV changed.

7.6 Comparison with an OS System Call

Aspect	Linux on a desktop	Bare-metal Cortex-M3
Restricted code	user-mode process	`main` after nPRIV = 1
Trap instruction	`syscall` (x86-64), `svc` (ARM)	`svc #0`
Privileged side	the kernel	`SVC_Handler`

Aspect	Linux on a desktop	Bare-metal Cortex-M3
Service	`write()` puts bytes on the terminal	print via semihosting

The mechanism is identical at every scale: the restricted side cannot do the job itself, so it traps into code that can, and that code decides what to do. This is the core of every operating system's protection model: restricted code cannot touch the machine directly and must go through a single, controlled entry point.

1 Peripherals and Buses of the MPS2-AN385 Platform

Before writing a driver, it is necessary to know the hardware it talks to: which board is emulated, which peripherals it contains, where they sit in the address space, and how the CPU reaches them.

1.1 CMSDK Hardware and CMSIS Software Conventions

Two Arm standards cover the two sides of a peripheral:

- **CMSDK** defines the hardware blocks,
- **CMSIS** defines how software names and accesses them.

Aspect	CMSDK	CMSIS
Full name	Cortex-M System Design Kit	Cortex Microcontroller Software Interface Standard
Nature	**hardware**: ready-made peripheral designs placed next to a Cortex-M core	**software**: a standard for the C headers and helper functions that describe a chip
Examples	APB UART, APB timer and dual timer, watchdog, AHB GPIO	`core_cm3.h`, `NVIC_EnableIRQ()`, `UART0->DATA`, `UART0TX_IRQn`
Key property	registers documented once by Arm, identical on every chip that integrates the block	shipped by chip vendors with their parts
In this context	the peripherals of the MPS2-AN385 board	a register header written by hand **in CMSIS style**, without vendor libraries, so names match real vendor code

1.2 Board, Clock, and Peripheral Address Map

The code runs on an emulated Arm development board. This section identifies the board, the single clock that times everything, and the addresses where its peripherals are located.

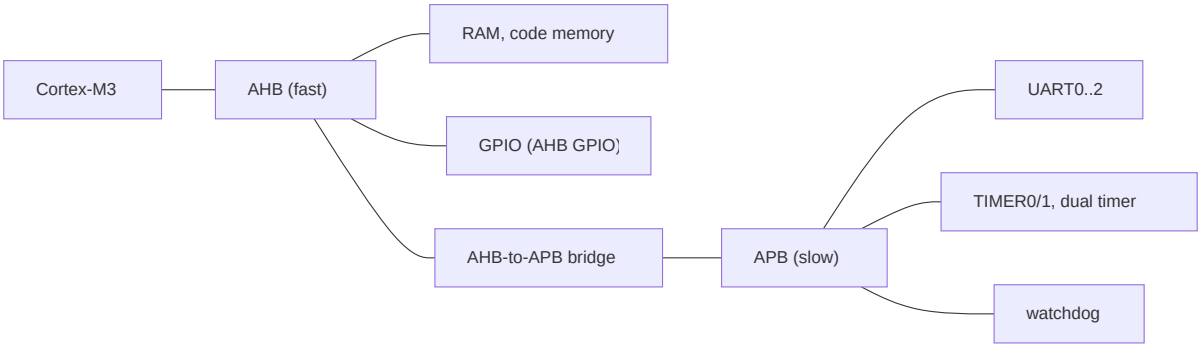
Element	Description
MPS2	Arm development board whose main chip is an **FPGA**: a reprogrammable chip whose circuit is loaded at power-on, so the same board can imitate different microcontrollers
AN385	the circuit (FPGA image, described in Arm Application Note 385) used here: a **Cortex-M3** CPU plus CMSDK peripherals
QEMU `mps2-an385`	emulator of exactly this system: the code runs on the PC without the physical board
Clock	one **25 MHz** clock shared by CPU and peripherals; every timing value (UART speed, timer period) is computed from it
Peripheral region	addresses `0x40000000` – `0x5FFFFFFF`, reserved by the Cortex-M memory map for devices; each device occupies a 4 KiB window



1.3 AHB and APB Buses

The CPU does not talk to every device in the same way: fast and slow devices sit on two different buses, connected by a bridge.

A **bus** carries every load and store from the CPU to a memory or a device.



Bus	Characteristics
AHB (Advanced High-performance Bus)	pipelined, one transfer per cycle: memories, GPIO
APB (Advanced Peripheral Bus)	simple, slow, low-power: rarely accessed registers (UART, timers, watchdog)
AHB-to-APB bridge	translates each access: a few extra cycles per APB register

For software **the bus is transparent**: only the timing differs. Software does not need to know which bus a device is on: every register access is the same load or store instruction, and the bridge routes it automatically. The only difference is speed: an APB access takes a few extra clock cycles.

1.4 Standard I/O Interfaces and DMA

Microcontrollers exchange data with the outside world through a small set of standard **interfaces**; a DMA controller can additionally move data without using the CPU.

Interface	Wires	Operating principle	Typical use
GPIO	1 per pin	each pin is a digital input (level read) or output (driven to 0 or 1)	buttons, LEDs, chip-selects
UART	2 (TX, RX)	asynchronous: no clock wire, both sides agree on a **baud rate**	debug console, PC link
SPI	clock, data out, data in, 1 select per device	synchronous; the controller drives the clock, one bit out and one in per tick; **chip-select** picks the device	flash, displays (MHz)
I2C	2 (clock, data)	synchronous multi-drop; the controller first sends the 7-bit device **address**	sensors (100–400 kHz)
ADC / DAC	1 per channel	analog voltage to number (ADC) or number to voltage (DAC)	temperature, audio

1.4.1 GPIO

A **GPIO** (General-Purpose Input/Output) pin is the simplest interface: a single wire whose level the software can read (input) or set to 0 or 1 (output). When a pin is used as an output, the electrical circuit that drives it determines whether the pin can safely share its wire with other devices.

Output type	Circuit	Level 1	Level 0	Shared wire
Push-pull	two transistors: one to the supply, one to ground	driven actively (connected to the supply)	driven actively (connected to ground)	**not allowed**: one device driving 1 and another driving 0 short-circuit supply and ground
Open-drain	one transistor to ground only	not driven: an external **pull-up resistor** brings the line high when nobody pulls it down	driven actively (connected to ground)	**allowed**: nobody ever drives 1, so there is no conflict; the line is 0 if at least one device pulls it low (wired-AND)

Push-pull is the normal choice for LEDs and chip-selects (fast, clean edges). Open-drain is required when several devices share one wire, as on the two I2C lines.

1.4.2 UART

A **UART** (Universal Asynchronous Receiver-Transmitter) sends bytes one bit at a time over a single wire per direction. It is **asynchronous**: there is no clock wire, so transmitter and receiver must agree in advance on the speed, the **baud rate** (bits per second), and each side times the bits with its own clock.

Element	Description
Wires	TX (transmit) and RX (receive), crossed between the two devices: the TX of one goes to the RX of the other; plus a common ground
Idle line	held at 1 when nothing is sent
Start bit	one 0 bit: the falling edge tells the receiver that a byte is starting and synchronizes its timing
Data bits	usually 8, sent **least significant bit first**
Parity bit	optional error check (none in the common **8N1** format: 8 data bits, no parity, 1 stop bit)
Stop bit	one 1 bit: returns the line to idle before the next byte

Transmission of `'A'` (`0x41` = `0100 0001`) in 8N1 format, bit 0 first:

```
idle  start  b0  b1  b2  b3  b4  b5  b6  b7  stop  idle
 1      0     1   0   0   0   0   0   1   0    1     1
```

Each byte therefore takes **10 bits** on the wire (start + 8 data + stop). At 115 200 baud one bit lasts about 8.7 µs, so one byte takes about **87 µs**: a long time for a CPU running at 25 MHz, which is why the way the CPU waits for the UART (polling or interrupts) matters.

1.4.3 DMA

Moving data between a peripheral and memory normally costs CPU time: the CPU must read each byte from the device and store it in memory, or the opposite. A **DMA controller** (Direct Memory Access) is a separate hardware unit that performs these copies on its own, leaving the CPU free to execute other code while the transfer is in

progress.

Aspect	Description
Operation	copies source → destination for a count, using the bus while the CPU runs; usually signals completion with an interrupt
Directions	peripheral → memory (UART RX, ADC); memory → peripheral (SPI TX)
Costs	bus contention; the buffer must not be touched during the transfer
AN385	no DMA: the CPU moves every byte

> **WARNING**
>
> In this course the CPU moves every byte.

2 Memory-Mapped I/O

On a Cortex-M, devices have **no special I/O instructions**: each device register is a **memory address**, and the CPU controls the device with ordinary loads and stores.

With **memory-mapped I/O** (MMIO), every peripheral register has a fixed physical address and is accessed with ordinary load and store instructions:

```
#define UART0_DATA (*(volatile uint32_t *)0x40004000u)

UART0_DATA = 'A';      /* a store to 0x40004000: the UART transmits 'A' */
```

The assignment compiles to a single `str` at `0x40004000`: the syntax is a variable access, the meaning is defined by the hardware.

2.1 Device Registers Are Not RAM

Although a device register is accessed like a variable, **it does not behave like memory:** reading or writing it can trigger actions in the hardware.

Property of a device register	Example
A **read** can have a side effect	consuming a received byte, acknowledging an event
A **write** can start an action	a transmission, a flag clear, a pin change
The value can **change without any store**	a status bit set when a byte arrives
The value read back can **differ** from the value written	read-only bits, self-clearing bits, reserved bits that must be written as zero or preserved

2.2 The `volatile` Qualifier

The C compiler assumes that **memory changes only when the program writes it**. For device registers this assumption is false, and `volatile` tells the compiler not to rely on it.

Under optimization the compiler caches values, removes repeated reads, and merges stores: all invalid for device registers.

TRY IT: `VOLATILE` AT `-O2`

Goal: see the compiler delete a hardware read. In a folder **outside** `base/` (e.g. `/tmp/vol`), create `poll.c`:

```c
#include <stdint.h>

#ifndef QUAL
#define QUAL    /* empty: a plain pointer */
#endif

/* UART0 STATE, bit 0 = TXBF */
#define UART0_STATE \
    (*(QUAL uint32_t *)0x40004004u)

void wait_tx_room(void)
{
    while (UART0_STATE & 1u)
    {
        /* wait until TXBF is 0 */
    }
}
```

Compile it twice at `-O2`, once with `-DQUAL=volatile` (defines `QUAL` as `volatile`), and disassemble both:

control the level of optimization

```
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 \
  -O2 -c poll.c -o plain.o
arm-none-eabi-gcc -mthumb -mcpu=cortex-m3 \
  -O2 -DQUAL=volatile -c poll.c -o vol.o
arm-none-eabi-objdump -d plain.o
arm-none-eabi-objdump -d vol.o
```

Predict: which version jumps back to the `ldr`? (Answers next slide.)

TRY IT: `VOLATILE` AT `-O2` — ANSWERS

`plain.o`: the read happens once; the "loop" is a jump to itself.

```
0: mov.w r3, #0x40004000   /* r3 = 0x40004000                        */
4: ldr   r3, [r3, #4]      /* r3 = STATE   (read ONCE)                */
6: lsls  r3, r3, #31       /* shift bit 0 into the sign: N flag = TXBF */
8: bmi.n c                 /* if (TXBF) goto c                        */
a: bx    lr                /* return                                  */
c: b.n   c                 /* c: goto c  -> for (;;) { }, never reads again */
```

`vol.o`: the loop goes back to the `ldr` and reads `STATE` on every turn.

```
0: mov.w r2, #0x40004000   /* r2 = 0x40004000                */
4: ldr   r3, [r2, #4]      /* r3 = STATE   (read every turn)  */
6: lsls  r3, r3, #31       /* N flag = TXBF                   */
8: bmi.n 4                 /* if (TXBF) goto 4: read again    */
a: bx    lr                /* return                          */
```

Without `volatile`, a UART busy at the first read hangs the program for ever. At `-O0` (our Makefile) the plain version happens to re-read, so the bug hides until you optimize: always use `volatile` for registers.

Build	Loop branch target	Effect
`-O2`, plain	`c` (itself)	`STATE` read once: a UART busy at the first read hangs the program forever
`-O2`, `volatile`	`4` (the `ldr`)	`STATE` re-read on every iteration: correct
`-O0`, plain	—	re-read by chance: the bug stays **latent** until optimization is enabled

> **WARNING**
>
> What `volatile` does and does not do It forces every source access to be performed, in order with respect to other `volatile` accesses. It gives **no atomicity** and **no synchronization** between `main` and a handler.

2.3 Read-Modify-Write Hazards

Changing one bit of a register with `|=` or `&=` looks like a single (atomic) operation in C, but the CPU performs it in several different steps. Since a device register can also change at run time, something can go wrong between them.

```
CMSDK_GPIO0->DATAOUT |= (1u << 0);      /* set bit 0 of the output latch */
```

```
    ldr  r1, [r0]       /* 1. READ   the whole register into r1   */
    orr  r1, r1, #1     /* 2. MODIFY bit 0 inside the CPU         */
    str  r1, [r0]       /* 3. WRITE  the whole value back         */
```

The GPIO (`0x40010000`) offers both ways of changing a pin:

Register	Function	How it is written
`DATA`	read: current pin levels; write: pin drive	plain store
`DATAOUT`	output latch: the levels driven on output pins	**read-modify-write** (OR-assignment or `&=`) to change one bit
`OUTENABLESET` / `OUTENABLECLR`	make pins outputs / inputs	**set/clear register**: only the 1 bits act, 0 bits are ignored

```
static void gpio_init(void)
{
    CMSDK_GPIO0->OUTENABLESET = (1u << 0);   /* one store: pin 0 becomes an output */
}

static void gpio_set(unsigned pin, int high)
{
    if (high) CMSDK_GPIO0->DATAOUT |= (1u << pin);    /* read-modify-write */
    else      CMSDK_GPIO0->DATAOUT &= ~(1u << pin);   /* read-modify-write */
}
```

`gpio_init` is safe: a single store, no read. `gpio_set` rewrites the **whole** register to change one bit, which fails in three cases:

Hazard	What happens	Example
Concurrent modification	another context changes the register between the read and the write; the write puts back the old value and **cancels** that change	`main` sets bit 0 of `DATAOUT`; an interrupt between `ldr` and `str` sets bit 1; the `str` writes the value read before, and bit 1 is lost
Write-1-to-clear (W1C) bits	writing a 1 *clears* a flag; writing back the read value clears **every** flag that was set	UART `INTSTATUS` reads `0b11` (TX and RX pending); OR-ing in TX writes `0b11` back and also clears RX: the RX event is lost
Read differs from write	the value read is not what was written; writing it back changes bits never meant to be touched	GPIO `DATA` returns the pin levels, including input pins; writing it back drives outputs with the levels just observed instead of the intended ones

The safe alternatives avoid the read, or make sure nothing can happen between read and write:

Safe pattern	Code	When
Set/clear register	`CMSDK_GPIO0->OUTENABLESET = (1u << pin);`	whenever the device provides one (GPIO `OUTENABLESET` / `OUTENABLECLR`, NVIC `ISER` / `ICER`)
Plain write of the single bit	`CMSDK_UART0->INTSTATUS = UART_INT_TX;`	W1C registers: only the written 1 acts, the other flags are untouched
Protected read-modify-write	`irq_save()`, read-modify-write, `irq_restore()` (6.4), or done before interrupts are enabled	unavoidable cases, e.g. `CCR`, where `STKALIGN` must stay 1

2.4 Register Overlays with C Structures

Instead of one macro per register, a whole register block can be described by a C `struct` placed at the device's base address, so that every register becomes a named field.

```
typedef struct
{
    volatile uint32_t DATA;      /* 0x00 read = received byte, write = byte to send */
    volatile uint32_t STATE;     /* 0x04 status flags */
    volatile uint32_t CTRL;      /* 0x08 control flags */
    volatile uint32_t INTSTATUS; /* 0x0C interrupt status; write 1 to clear a bit */
    volatile uint32_t BAUDDIV;   /* 0x10 baud-rate divider */
} CMSDK_UART_TypeDef;

#define CMSDK_UART0   ( ( CMSDK_UART_TypeDef * ) 0x40004000UL )
```

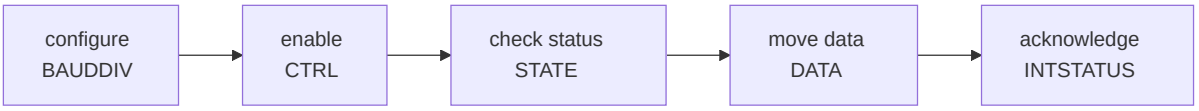
Element	Meaning
Layout	fields at consecutive 4-byte offsets from the base address

Element	Meaning
`CMSDK_UART0`	a constant pointer, not a variable: `->STATE` is a load from `0x40004004`
Safety	`volatile` fields; bit names next to the `struct` (`UART_STATE_TXBF` is `1UL << 0`); `_Static_assert` + `offsetof` catch layout typos at compile time

The same technique describes SysTick, NVIC, SCB, timers, watchdog, GPIO, and FPGAIO.

2.5 Registers as a Hardware Protocol

Device registers must be used in a precise order, like the steps of a protocol; the **driver is the code that implements this order**.



Rule	Example
Configuration precedes enabling	speed before switching the transmitter on
A status check precedes every transfer	write `DATA` only when there is room, read it only when a byte is present
An event is acknowledged	clearing its flag lets the device report the next one

2.6 Real, Placeholder, and Missing Devices under QEMU

QEMU **does not emulate every device of the board equally well**. Knowing which devices are real models and which are placeholders explains why some drivers cannot be tested in the emulator.

The QEMU monitor lists the object answering at each address range:

```
printf 'info mtree -f\nquit\n' | qemu-system-arm -M mps2-an385 -display none -S -monitor stdio
-serial null | grep ' 00000000400' | sort -u
```

```
0000000040000000-0000000040000fff (prio 0, i/o): cmsdk-apb-timer
0000000040002000-0000000040002fff (prio 0, i/o): cmsdk-apb-dualtimer
0000000040004000-0000000040004fff (prio 0, i/o): uart
0000000040008000-0000000040008fff (prio 0, i/o): cmsdk-apb-watchdog
0000000040010000-0000000040010fff (prio -1000, i/o): cmsdk-ahb-gpio
0000000040020000-0000000040020fff (prio 0, i/o): pl022
0000000040022000-0000000040022fff (prio 0, i/o): arm_sbcon_i2c
0000000040028000-0000000040028fff (prio 0, i/o): mps2-fpgaio
```

Case	Recognized by	Behaviour	Example	`-d unimp` output
Real model	`prio 0` + device name	registers behave like the hardware	UART0 `0x40004000`, SysTick, NVIC	nothing

Case	Recognized by	Behaviour	Example	`-d unimp` output
Placeholder	`prio -1000`	writes ignored, reads return 0	GPIO0 `0x40010000`	line naming the device (`cmsdk-ahb-gpio`)
No device	no entry	—	`0x40080000`	line naming a region (`RESERVED 4`)

None of the cases faults. `-d unimp` logs accesses to placeholders and unmapped regions, `-d guest_errors` misuse of real models (e.g. writing to a disabled UART transmitter): a run with no `-d unimp` output places any malfunction in the code. A GPIO driver cannot be verified in QEMU; the FPGAIO LED register instead keeps the written value.

> **INFO**
>
> Core versus board peripherals **Core** peripherals (SysTick, NVIC) have addresses fixed by ARMv7-M and exist on every Cortex-M. **Board** peripherals (UART0, GPIO0, FPGAIO) exist where the board places them, modelled as faithfully as the emulator allows.

3 Device Drivers and Polled UART I/O

The first driver built on these concepts is the UART, the serial port used as console. On top of it, a small formatting library provides `kprintf` for kernel output.

3.1 CMSDK UART Register Interface

The UART (Universal Asynchronous Receiver-Transmitter) sends and receives bytes on a serial line. It is controlled through five registers at `0x40004000`.

Offset	Register	Access	Content
`0x00`	`DATA`	R/W	write: byte to send; read: received byte (clears RXBF)
`0x04`	`STATE`	R; bits 2–3 W1C	TXBF, RXBF, TXOR, RXOR
`0x08`	`CTRL`	R/W	TXEN, RXEN, TXIRQEN, RXIRQEN, overrun IRQ enables
`0x0C`	`INTSTATUS`	R/W1C	TX, RX, TXOR, RXOR interrupt pending
`0x10`	`BAUDDIV`	R/W	clock / baud rate, minimum 16

STATE 8-bit

UART STATE (0x04), low byte of the 32-bit register



Bits	Field	Description
3	RXOR	Receive overrun: a byte was lost (write 1 to clear)
2	TXOR	Transmit overrun: a write was lost (write 1 to clear)
1	RXBF	Receive buffer full: a byte is waiting
0	TXBF	Transmit buffer full: the previous byte is still leaving

CTRL 8-bit = 0x03

UART CTRL (0x08) after uart_init, low byte



Bits	Field	Value	Description
5	RXORIRQEN	0	Receive-overrun interrupt enable
4	TXORIRQEN	0	Transmit-overrun interrupt enable
3	RXIRQEN	0	Receive interrupt enable
2	TXIRQEN	0	Transmit interrupt enable
1	RXEN	1	Receiver enable
0	TXEN	1	Transmitter enable

INTSTATUS 8-bit

UART INTSTATUS (0x0C), write 1 to clear a bit



Bits	Field	Description
3	RXOR	Receive-overrun interrupt pending
2	TXOR	Transmit-overrun interrupt pending
1	RX	Receive interrupt pending
0	TX	Transmit interrupt pending

Behaviour	Consequence
One byte buffered per direction	a write while TXBF = 1 is lost (TXOR); a byte arriving while RXBF = 1 is lost (RXOR)
`CTRL` = 0 after reset	transmitter and receiver off: writes to `DATA` are ignored
`BAUDDIV` on real hardware	25 000 000 / 115 200 ≈ 217 for 115 200 baud
`BAUDDIV` under QEMU	speed ignored; 16 (minimum legal value) is used

3.2 The Device Driver Abstraction

A driver separates *what* the rest of the system wants to do (send a byte) from *how* a particular device does it (which registers to touch, in which order). It is the only code that touches the device's registers; everything else calls its functions.

All character devices (UART, terminal, …) offer the same three operations. They can be collected in a `struct` whose fields are **pointers to functions**: each field holds the address of the function that performs that operation for one specific device.

```
struct char_driver
{
    void ( *init )( void );       /* bring the device up     */
    int  ( *read )( void );       /* one byte, or -1 if none */
    void ( *write )( char c );    /* send one byte           */
};

/* The UART driver fills the struct with its own functions */
static const struct char_driver uart0_driver = {
    uart_init,                    /* init  */
    uart_getc_polled,             /* read  */
    uart_putc_polled              /* write */
};

/* Generic code: works with any driver passed to it */
void say_hi( const struct char_driver *dev )
{
    dev->write( 'h' );            /* calls the write function of that driver */
    dev->write( 'i' );
}
```

`say_hi( &uart0_driver )` prints "hi" on the UART; passing another driver prints it on another device, without changing `say_hi`.

Aspect	Description
Interface	every driver offers the same operations (init, read, write); replacing the UART means replacing one driver file
Generalization	operating systems use this pattern for every class of device: Linux `struct file_operations` (`.open`, `.read`, `.write`, …) is filled in by each driver
This kernel	with a single console the `struct` is not needed: the kernel calls `uart_putc_polled` directly

3.3 Polled UART Driver

The simplest UART driver uses **polling**: the CPU repeatedly reads the status register until the device is ready, **without using interrupts**.

```
#include "drivers/uart.h"
#include "cmsdk.h"

void uart_init( void )
{
    CMSDK_UART0->BAUDDIV = 16;                          /* configure */
    CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;    /* enable */
}

void uart_putc_polled( char c )
{
    while ( CMSDK_UART0->STATE & UART_STATE_TXBF )          /* check status: room? */
    {
    }
    CMSDK_UART0->DATA = (uint32_t) (unsigned char) c;       /* move data */
}

int uart_getc_polled( void )
{
    if ( ( CMSDK_UART0->STATE & UART_STATE_RXBF ) == 0 )    /* check status: byte? */
    {
        return -1;
    }
    return (int) ( CMSDK_UART0->DATA & 0xFF );              /* reading DATA takes the byte */
}
```

Function	Protocol steps	Behaviour
`uart_init`	configure (`BAUDDIV`), enable (`CTRL`)	brings the UART to a known state
`uart_putc_polled`	check TXBF in a loop, write `DATA`	blocks until there is room

Function	Protocol steps	Behaviour
`uart_getc_polled`	check RXBF once, read `DATA`	never blocks: returns `-1` if no byte is waiting

Polled: status is checked by the CPU, not signalled by interrupts. The cast through `unsigned char` keeps a byte such as `0xE9` from becoming `0xFFFFFFE9` when `char` is signed.

> ⚠ **WARNING**
>
> Enable before moving data Without `uart_init`, `CTRL` stays 0 and every byte is silently dropped (`-d guest_errors` : `transmit data write with Tx disabled`).

3.4 Variadic Functions and `<stdarg.h>`

A `printf`-like function accepts a variable number of arguments. C supports this with *variadic functions* and the macros of `<stdarg.h>`.

```
void kprintf( const char *fmt, ... )
{
    va_list ap;                     /* a cursor over the extra arguments    */
    va_start( ap, fmt );            /* start right after the last named one */
    int n = va_arg( ap, int );      /* take the next argument, as an int    */
    va_end( ap );                   /* done                                 */
}
```

Rule	Meaning
Argument count and types	unknown to the function: the **format string** supplies them; a mismatch is undefined behaviour
Default promotions	a `char` arrives as an `int`: `%c` uses `va_arg( ap, int )`
Freestanding	`<stdarg.h>` has only types and built-ins, no library code
Convention	a function taking a `va_list` is named `v…` (`vprintf`)

3.5 A Shared Formatting Core: `vformat` and `kprintf`

Turning a format string and its arguments into characters is the same work whatever the destination (UART, buffer, …), so it is written once in `vformat` and reused.

Formatting is separated from output: `vformat` calls `out( c, ctx )` per character, and `ctx` carries the destination's state.

```
static void put_string( void ( *out )( char c, void *ctx ), void *ctx, const char *s )
{
    while ( *s != '\0' )
    {
```

```
        out( *s, ctx );
        s++;
    }
}

void vformat( void ( *out )( char c, void *ctx ), void *ctx, const char *fmt, va_list ap )
{
    char num[12];   /* longest number + '\0' */

    while ( *fmt != '\0' )
    {
        if ( *fmt != '%' )              /* ordinary character: pass it on */
        {
            out( *fmt, ctx );
            fmt++;
            continue;
        }
        fmt++;                          /* a '%': look at the next letter */
        switch ( *fmt )
        {
        case 'c':
            out( (char) va_arg( ap, int ), ctx );   /* a char arrives as an int */
            break;
        case 's':
        {
            const char *s = va_arg( ap, const char * );
            put_string( out, ctx, s != 0 ? s : "(null)" );  /* no crash on null */
            break;
        }
        case 'd':
        {
            int v = va_arg( ap, int );
            uint32_t mag;
            if ( v < 0 ) { out( '-', ctx ); mag = 0u - (uint32_t) v; }
            else         { mag = (uint32_t) v; }
            utoa_dec( mag, num );
            put_string( out, ctx, num );
            break;
        }
        case 'u': utoa_dec( va_arg( ap, uint32_t ), num ); put_string( out, ctx, num ); break;
        case 'x': utoa_hex( va_arg( ap, uint32_t ), num ); put_string( out, ctx, num ); break;
        case '%': out( '%', ctx ); break;
        case '\0': out( '%', ctx ); return;          /* string ended right after '%' */
```

```
        default:  out( '%', ctx ); out( *fmt, ctx ); break;  /* unknown: as written */
        }
        fmt++;                          /* past the conversion letter */
    }
}
```

Aspect	Description
Conversions	`%c` , `%s` , `%d` , `%u` , `%x` (no `0x`), `%%` ; no width; unknown ones printed literally
`%d` magnitude	`0u - (uint32_t) v` , not `-v` : `-INT_MIN` overflows (UB), unsigned subtraction is always defined

```
static void kprintf_out( char c, void *ctx )
{
    (void) ctx;                         /* not used */
    uart_putc_polled( c );
}

void kprintf( const char *fmt, ... )
{
    va_list ap;
    va_start( ap, fmt );
    vformat( kprintf_out, 0, fmt, ap );
    va_end( ap );
}
```

`kprintf( "d=%d\n", -42 )` makes six `kprintf_out` calls. Polled, unbuffered, and interrupt-free, `kprintf` works in **any context**, fault handlers included.

3.6 Fault Handlers That Report CFSR and HFSR

With a working `kprintf` , fault handlers can print *why* a fault happened instead of hanging silently, by **reading the fault status registers of the core**.

Each fault handler prints its status register with `kprintf` , then halts.

Register	Address	Content
CFSR (Configurable Fault Status)	`0xE000ED28`	one bit per cause of MemManage, BusFault, UsageFault
HFSR (HardFault Status)	`0xE000ED2C`	why a HardFault occurred

EXAMPLE

Division by zero With `CCR.DIV_0_TRP` set, an integer division by a `volatile` zero (`volatile` prevents compile-time evaluation) prints `UsageFault: CFSR=0x2000000` : bit 25, `DIVBYZERO` .

TRY IT: FAULT HANDLERS THAT PRINT (1/3)

Goal: the base's fault handlers are silent loops; make them print through the UART. In `kernel/faults.c`, replace everything below the header comment with the code of the two columns, left then right (the same four handlers, now printing, plus a `halt` helper):

```c
#include "cmsdk.h"
#include "kernel/kprintf.h"
/* Stop here for ever. */
static void halt( void )
{
    for ( ;; )
    {
    }
}
/* HFSR tells why it came here. */
void HardFault_Handler( void )
{
    kprintf( "HardFault: HFSR=0x%x\n", SCB->HFSR );
    halt();
}
```

```c
void MemManage_Handler( void )
{
    kprintf( "MemManage: CFSR=0x%x\n", SCB->CFSR );
    halt();
}
void BusFault_Handler( void )
{
    kprintf( "BusFault: CFSR=0x%x\n", SCB->CFSR );
    halt();
}
void UsageFault_Handler( void )
{
    kprintf( "UsageFault: CFSR=0x%x\n", SCB->CFSR );
    halt();
}
```

4 Interrupts, the AAPCS, and the Exception Model

Polling wastes CPU time; interrupts let a device call the CPU only when needed. This part explains how a Cortex-M enters and leaves an interrupt handler without corrupting the interrupted code.

4.1 Polling versus Interrupts

There are two ways to find out when a device is ready: the CPU keeps asking (**polling**), or the device signals the CPU (**interrupt**). Each has its uses.

Aspect	Polling	Interrupts
CPU time	spent in the waiting loop even when nothing happens	spent only per event (entry, handler, exit)
Latency	up to one trip around the loop; grows with the devices checked	about 12 cycles of entry on Cortex-M3, nearly constant
Complexity	one loop	handler, shared data, priorities
CPU between events	busy	free, or asleep (`wfi`)
Suited to	certain, imminent events; contexts without interrupts (fault handlers)	rare or unpredictable events, long transfers

One byte at 115 200 baud lasts ~87 µs, about **2 170 cycles** at 25 MHz wasted by polling.

4.2 AAPCS Register Roles

The **AAPCS** (Procedure Call Standard for the Arm Architecture) is the **set of rules** that every function call follows on ARM, in C and in assembly alike: which registers carry arguments and results, and which registers a function must preserve. Interrupt handling relies on these rules.

Register	Role in a call	Preserved across the call
`r0` – `r3`	first four arguments; `r0` also returns the result	no: **caller-saved**
`r4` – `r11`	long-lived values of the function	yes: **callee-saved** (e.g. `push {r4, lr}` … `pop {r4, pc}`)
`r12`	scratch	no: caller-saved
`lr`	return address written by `bl`	no: a non-leaf function saves it
`sp`	multiple of 8 at every public call	yes

- **Caller-saved**: if the caller still needs r0–r3 or r12 after the call, it saves them itself
- **Callee-saved**: a function that wants r4 must restore it before returning (`push {r4, lr}` … `pop {r4, pc}`)

AAPCS: A CALL, STEP BY STEP

Calling `add3` and the code of `add3` itself:

```
int add3(int a, int b, int c);          add3:
r = add3(1, 2, 3);                          adds r0, r0, r1   /* r0 = a + b */
                                            adds r0, r0, r2   /* r0 = r0 + c */
movs r0, #1    /* a = 1 */                  bx   lr           /* return r0 to the caller */
movs r1, #2    /* b = 2 */
movs r2, #3    /* c = 3 */
bl   add3      /* lr = next instruction; pc = add3 */
               /* r = r0 */
```

- `add3` uses only r0–r2 and calls nothing, so it saves nothing: no stack at all
- A function that calls another must first save `lr` (`push {r4, lr}`), or `bl` would overwrite its own return address

`add3` is a leaf function using only caller-saved registers: it saves nothing.

4.3 Exception Entry and the Stacked Frame

When an interrupt (an exception) occurs, the hardware itself **saves part of the CPU state on the stack** before running the handler. This saved block is the *stacked frame*.

Before the first handler instruction, the hardware:

1. pushes **8 words** on the current stack (the stacked frame);
2. writes `EXC_RETURN` into `lr`;
3. fetches the handler address from the vector table and branches to it in Handler mode.

The frame, as observed in gdb at the start of `SysTick_Handler` (5.6): entry `[N]` is at `sp + 4N`.

Stacked frame on exception entry (values from gdb in 5.6)

0x203F_FFEC	[7] **xPSR** = 0x01000000 only the Thumb bit set
0x203F_FFE8	[6] **PC** = 0x00000510 where the interrupted code resumes
0x203F_FFE4	[5] **LR** = 0x0000050D interrupted code's lr
0x203F_FFE0	[4] **r12** = 0x00000000
0x203F_FFDC	[3] **r3** = 0x00000000
0x203F_FFD8	[2] **r2** = 0x00000007 (value just written to SysTick CTRL)
0x203F_FFD4	[1] **r1** = 0x00000000
0x203F_FFD0	[0] **r0** = 0x00000000 <- sp when the handler starts

The hardware saves exactly the AAPCS **caller-saved** set plus `PC` and `xPSR`, and any C function restores `r4` – `r11`: a handler is therefore an **ordinary C function**. `CCR.STKALIGN` aligns `sp` to 8 bytes on entry, as the AAPCS requires.

4.4 Exception Return through `EXC_RETURN`

A handler ends with an ordinary return instruction, yet the CPU must **restore** the saved state and resume the interrupted code. A special value in `lr`, `EXC_RETURN`, makes this possible.

In a handler `lr` holds `0xFFFFFFxx` (no code there): `bx lr` or `pop {..., pc}` becomes an **exception return** (pop 8 words, restore mode, resume at stacked `PC`).

`EXC_RETURN`	Return to	Stack
`0xFFFFFFF9`	Thread mode	main (MSP)
`0xFFFFFFF1`	Handler mode (a preempted handler)	main (MSP)
`0xFFFFFFFD`	Thread mode	process (PSP)

When you return from an exception, the Linker Register is not containing the return address but also a special code 0xF

In which execution mode the CPU should go back

EXCEPTION RETURN: `EXC_RETURN`

In a handler, `lr` does not hold a code address: it holds a value starting with `0xFFFFFF..`, where there is no code. When the handler returns with `bx lr` (or `pop {..., pc}`), the CPU sees this value and does an **exception return**: it pops the 8 words, goes back to the old mode, and resumes at the stacked PC.

`EXC_RETURN`	Return to	Stack used
`0xFFFFFFF9`	thread mode (normal code, e.g. `main`)	main stack
`0xFFFFFFF1`	handler mode (a less urgent handler that was interrupted)	main stack
`0xFFFFFFFD`	thread mode	process stack

- Cortex-M has a **second stack pointer** (the process stack). This kernel never uses it, so a return to thread mode here is always `0xFFFFFFF9`
- The compiled C handler ends with an ordinary `bx lr` or `pop {..., pc}`: the value in `lr` is what makes it an exception return

EXC_RETURN 32-bit **= 0xFFFFFFF9**
Thread mode, main stack (MSP)



Bits	Field	Value	Description
31:4	`PREFIX`	0xFFFFFFF (268435455)	All ones: marks an EXC_RETURN value
3	`MODE`	1 → Thread	Return mode 0 = Handler, 1 = Thread
2	`SPSEL`	0 → MSP	Return stack 0 = MSP, 1 = PSP

EXC_RETURN 32-bit **= 0xFFFFFFF1**
Handler mode (a preempted handler), MSP



Bits	Field	Value	Description
31:4	`PREFIX`	0xFFFFFFF (268435455)	All ones: marks an EXC_RETURN value
3	`MODE`	0 → Handler	Return mode 0 = Handler, 1 = Thread
2	`SPSEL`	0 → MSP	Return stack 0 = MSP, 1 = PSP

EXC_RETURN 32-bit **= 0xFFFFFFFD**
Thread mode, process stack (PSP)



Bits	Field	Value	Description
31:4	`PREFIX`	0xFFFFFFF (268435455)	All ones: marks an EXC_RETURN value
3	`MODE`	1 → Thread	Return mode 0 = Handler, 1 = Thread
2	`SPSEL`	1 → PSP	Return stack 0 = MSP, 1 = PSP

4.5 Tail-Chaining and Late Arrival

When several exceptions occur close together, the Cortex-M avoids saving and restoring the same state repeatedly, with two hardware optimizations.

Optimization	Situation	Behaviour
Tail-chaining	a handler ends while another exception is pending	no pop and re-push of the 8 words: direct jump to the next handler (~6 cycles instead of ~24)
Late arrival	a more urgent exception arrives while the frame of a less urgent one is being pushed	the frame is reused, the more urgent handler runs first, then tail-chains to the other

5 The SysTick Timer and Wraparound-Safe Time Keeping

An operating system needs a notion of time. SysTick, a **timer** built into every Cortex-M, generates a **periodic interrupt** (the *tick*) from which the kernel counts **milliseconds**.

5.1 SysTick Registers

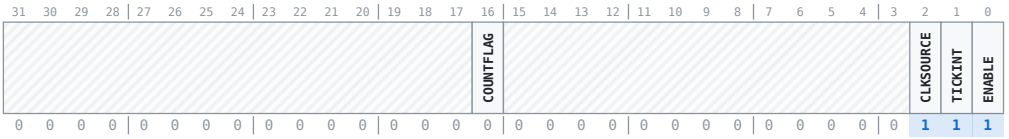
SysTick is configured through four registers at fixed addresses, identical on every Cortex-M.

SysTick is a 24-bit down-counter present in every Cortex-M core: it counts from `LOAD` to 0 and reloads, so **one period is `LOAD + 1` cycles**.

Register	Address	Content
`CTRL`	`0xE000E010`	ENABLE, TICKINT (exception at 0), CLKSOURCE (1 = processor clock), COUNTFLAG (cleared on read)
`LOAD`	`0xE000E014`	24-bit reload value (max `0xFFFFFF`)
`VAL`	`0xE000E018`	current count; any write clears it to 0
`CALIB`	`0xE000E01C`	read-only: 10 ms reload value for the reference clock (CLKSOURCE = 0)

CTRL 32-bit = **0x00000007**

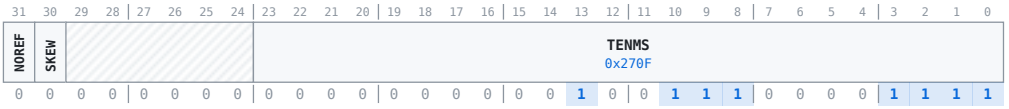
SysTick CTRL (0xE000E010) after systick_init



Bits	Field	Value	Description
16	`COUNTFLAG`	0	Counter reached 0 since the last read (cleared on read)
2	`CLKSOURCE`	1 → processor clock	Clock source 0 = reference clock, 1 = processor clock
1	`TICKINT`	1	Raise the SysTick exception at 0
0	`ENABLE`	1	Counter running

CALIB 32-bit = **0x0000270F**

SysTick CALIB (0xE000E01C) on the AN385 model



Bits	Field	Value	Description
31	`NOREF`	0 → reference clock present	No reference clock 0 = reference clock present, 1 = none
30	`SKEW`	0 → exact	TENMS not exactly 10 ms 0 = exact, 1 = inexact
23:0	`TENMS`	0x270F (9999)	Reload value for 10 ms on the reference clock

5.1.1 Choice of the Clock Source

SysTick can count with one of two clocks, selected by the CLKSOURCE bit of `CTRL`.

	Reference clock	Processor clock
Selected by	CLKSOURCE = 0	CLKSOURCE = 1
Frequency on AN385	1 MHz	25 MHz
How `LOAD` is obtained	read from `CALIB.TENMS` (ready-made value for 10 ms)	computed by the kernel: `LOAD` = f / rate − 1 (5.2)
Used here	no	**yes**

Why the reference clock is 1 MHz:

- `CALIB.TENMS` = 9999 is the reload value for a 10 ms period;
- one period lasts `TENMS + 1` = **10 000 ticks**;
- 10 000 ticks in 10 ms = 1000000 ticks per second = **1 MHz**.

`CALIB` is useful to portable code that does not know the processor frequency. Here the frequency is known (25 MHz), so the kernel ignores `CALIB` and uses the processor clock.

5.1.2 SysTick as an exception

SysTick is a **core exception** (number 15), not a device interrupt routed through the NVIC:

Aspect	SysTick (core exception)	Device interrupt (e.g. UART TX)
Priority set in	`SHPR` (System Handler Priority Registers): `0xE0`	NVIC `IPR`
Enabled by	`CTRL.TICKINT`	device enable bit + NVIC `ISER`
Pending state	cleared automatically when the handler starts	stays set until the handler clears the device flag
Acknowledge in the handler	**not needed**	**required**, otherwise the handler is re-entered

5.2 Reload Value and Tick Rate

The tick period is chosen by writing a reload value; it is computed from the clock frequency and the desired number of ticks per second.

From period = (`LOAD` + 1) / f: **`LOAD` = f / rate − 1**. At 25 MHz and 100 ticks/s, `LOAD` = 25 000 000 / 100 − 1 = **249 999** (`0x3D08F`), which fits in 24 bits; the longest period is 2^24 cycles ≈ 0.67 s.

Tick	Rate	Resolution	Exceptions/s	Note
1 ms	1 000 Hz	1 ms	1 000	in QEMU runs about 26% slow in wall-clock time
10 ms	100 Hz	10 ms	100	classic OS rate; exact wall-clock time in QEMU

A faster tick gives finer resolution at the cost of more exceptions.

5.3 SysTick Driver and Uptime Counter

The driver starts SysTick and counts elapsed milliseconds in a variable updated by the interrupt handler.

```
#define CPU_CLOCK_HZ 25000000UL     /* processor clock: 25 MHz */

static volatile uint32_t uptime_ms; /* changed by the handler */

void systick_init( void )
{
    uptime_ms = 0;
    SysTick->LOAD = CPU_CLOCK_HZ / ( 1000 / TICK_MS ) - 1;   /* 249 999 for TICK_MS = 10 */
    SysTick->VAL = 0;               /* any write: counter = 0 */
    SysTick->CTRL = SysTick_CTRL_ENABLE | SysTick_CTRL_TICKINT | SysTick_CTRL_CLKSOURCE;
```

```
}

void SysTick_Handler( void )
{
    uptime_ms += TICK_MS;           /* no acknowledge needed */
}

uint32_t systick_uptime_ms( void )
{
    return uptime_ms;               /* one 32-bit read: never torn */
}
```

`SysTick_Handler` overrides the weak default in slot 15; `uptime_ms` is `volatile` because the handler changes it, and its single aligned 32-bit read can never be **torn**.

5.4 Wraparound-Safe Elapsed Time

The kernel measures time with a `uint32_t` counter of milliseconds (`uptime_ms`). A 32-bit variable can count only up to $2^{32}-1$ (`0xFFFFFFFF`); one millisecond later it **wraps around** to 0. This happens after 2^{32} ms ≈ **49.7 days**, a realistic uptime for an embedded device, so code that measures elapsed time must stay correct across the wrap.

> **ANALOGY: A 12-HOUR CLOCK**
>
> From 11 o'clock to 2 o'clock, 3 hours pass, even though 2 is smaller than 11. Counting on the clock face, $(2-11) \bmod 12 = 3$. Unsigned arithmetic in C works the same way, on a "clock" of 2^{32} positions: subtraction is computed **modulo** 2^{32}, so `now - start` is always the true distance from `start` to `now` , even if `now` has wrapped past 0.

Two ways to test whether `delay` ms have passed since `start` :

Test	Across the wrap	Why
`now - start >= delay`	**correct**	the subtraction gives the real elapsed time, modulo 2^32
`now >= start + delay`	**wrong**	`start + delay` can itself wrap to a small number, so the comparison becomes true too early

WORKED EXAMPLE: `START = 0XFFFFFF00`, `DELAY = 300` MS

`start` is 256 ms before the wrap (`0xFFFFFFFF` + 1 = 0).

Moment	`now`	`now - start` (correct test)	`start + delay` (wrong test)
16 ms after start	`0xFFFFFF10`	16 → 16 < 300: **not yet** ✔	wraps to 44; `0xFFFFFF10` ≥ 44 → **elapsed ✘** (284 ms too early)
356 ms after start (after the wrap)	`0x00000064` (100)	256 + 100 = 356 → 356 ≥ 300: **elapsed ✔**	—

The subtraction always measures the real distance; the addition breaks as soon as `start + delay` passes `0xFFFFFFFF`.

5.5 Idle Loop with `wfi`

When there is nothing to do, the CPU should sleep instead of spinning. The `wfi` instruction puts it to **sleep until the next interrupt.**

```
uint32_t last = systick_uptime_ms();
for ( ;; )
{
    __asm volatile ( "wfi" );           /* sleep until an interrupt */
    uint32_t now = systick_uptime_ms();
    if ( now - last >= 1000 )           /* safe across wraparound */
    {
        last += 1000;
        kprintf( "uptime: %u ms\n", now );
    }
}
```

Element	Effect
`wfi`	sleeps until an exception is pending; the loop runs once per interrupt, at least every 10 ms
`last += 1000`	schedules each print 1 s after the previous one: no drift (unlike `last = now`)

5.6 Inspecting the Exception Frame in gdb

The stacked frame described in 4.3 can be observed directly with gdb, by stopping at the first instruction of `SysTick_Handler`.

```
(gdb) break *SysTick_Handler        # stop at the very first instruction
(gdb) continue
(gdb) x/8wx $sp                     # r0 r1 r2 r3 / r12 lr pc xPSR
```

```
0x203fffd0:  0x00000000  0x00000000  0x00000007  0x00000000
0x203fffe0:  0x00000000  0x0000050d  0x00000510  0x01000000
(gdb) p/x $lr
$1 = 0xfffffff9
(gdb) info symbol 0x510
main + 28 in section .text
```

Observation	Meaning
`break *SysTick_Handler`	stops before the prologue, while `sp` points at the frame
Stacked `PC` = `0x510`	instruction after `wfi` in `main`
Stacked `r2` = 7	value just written to `CTRL`
`lr` = `0xFFFFFFF9`	return to Thread mode, MSP

6 The NVIC and Interrupt-Driven UART Transmission

Device interrupts are managed by the **NVIC** (Nested Vectored Interrupt Controller). Using it, the UART can transmit in the background: the program puts bytes in a buffer, and an interrupt handler sends them one by one.

6.1 NVIC Registers and IRQ Numbering

Device interrupts do not reach the CPU directly: they pass through the **NVIC**, which numbers them, enables them, and assigns their priority.

Term	Meaning
NVIC (Nested Vectored Interrupt Controller)	the part of the core, at `0xE000E100`, that receives interrupt requests from devices, enables or disables them, and decides their priority
IRQ (Interrupt ReQuest)	a numbered line through which a device asks the CPU for attention; AN385 has 32 of them, **IRQ 0–31**

Every IRQ is also an **exception**, with its own slot in the vector table. The first 16 exception numbers are taken by the core, so device interrupts come after them:

Exception number	Belongs to	Examples
0–15	the **core**	reset, HardFault, SVCall (11), SysTick (15)
16–47	**devices**: IRQ 0–31	UART0 RX (IRQ 0), UART0 TX (IRQ 1), TIMER0 (IRQ 8)

From the IRQ number to the handler:

- **exception number** = 16 + n
- **vector slot** = 16 + n
- **slot address** = 4 × (16 + n), since each slot holds a 4-byte handler address

UART0 TX

IRQ 1 → exception 16 + 1 = 17 → slot address 4 × 17 = 68 = `0x44`, which holds the address of `UART0TX_Handler`.

IRQ	Source	IRQ	Source
0	UART0 RX	8	TIMER0
1	UART0 TX	9	TIMER1
2 / 3	UART1 RX / TX	10	dual timer
4 / 5	UART2 RX / TX	12	UART overflow (all UARTs; never fires in QEMU)

Vector table with device interrupts (48 words, 0xC0 bytes)

0x0000_00BC	[47] **IRQ31** last NVIC line
	⋮ 18 entries (29–46)
0x0000_0070	[28] **UART_OVERFLOW** IRQ 12: all UARTs combined
	⋮ 1 entry (27)
0x0000_0068	[26] **DUALTIMER** IRQ 10
0x0000_0064	[25] **TIMER1** IRQ 9
0x0000_0060	[24] **TIMER0** IRQ 8
	⋮ 2 entries (22–23)
0x0000_0054	[21] **UART2_TX** IRQ 5
0x0000_0050	[20] **UART2_RX** IRQ 4
0x0000_004C	[19] **UART1_TX** IRQ 3
0x0000_0048	[18] **UART1_RX** IRQ 2
0x0000_0044	[17] **UART0TX_Handler** IRQ 1: UART0 TX
0x0000_0040	[16] **UART0_RX** IRQ 0
0x0000_003C	[15] **SysTick_Handler** SysTick
	⋮ 3 entries (12–14)
0x0000_002C	[11] **SVC_Handler** SVCall
	⋮ 8 entries (3–10)
0x0000_0008	[2] **NMI_Handler** Watchdog expiry
0x0000_0004	[1] **Reset_Handler**
0x0000_0000	[0] **_estack** Initial SP

The watchdog raises the non-maskable **NMI** (Non-Maskable Interrupt). Device handlers are `.weak` aliases of `Default_Handler`, replaced by a C function with the same name.

- **Watchdog**: a timer that the software must reload periodically. If the program hangs and stops reloading it, the watchdog fires, so the system can recover from a lock-up.
- **NMI** (Non-Maskable Interrupt, exception 2): an interrupt that **cannot be disabled**.
  - `PRIMASK` does not block it, and its fixed priority is above every configurable one.
  - The watchdog uses it because a hung program may have masked all ordinary interrupts.
- **Weak handlers**: every handler name in the vector table is a `.weak` alias of `Default_Handler` (an infinite loop).
  - A C function with the same name (e.g. `UART0TX_Handler`) replaces it at link time, without editing the table.

Register	Address	Bit / byte for IRQ n	Write semantics
`ISER`	`0xE000E100`	bit n % 32 of word n / 32	1 = enable, 0 = no change
`ICER`	`0xE000E180`	same	1 = disable
`ISPR`	`0xE000E200`	same	1 = set pending
`IPR`	`0xE000E400`	byte n	priority; only upper bits implemented; lower = more urgent

```
NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* priority of IRQ 1 */
NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
```

Set/clear registers ignore 0 bits: a plain assignment suffices, no `|=`.

6.2 Enabling and Acknowledging a Device Interrupt

For a device interrupt to reach its handler, several switches must be on; once in the handler, the interrupt must be **acknowledged** or **it fires again forever**.

A device interrupt runs its handler only if all three conditions hold:

1. the **device** may raise it (UART TX: `CTRL.TXIRQEN`, bit 2);
2. the **NVIC** line is enabled in `ISER`;
3. its priority is more urgent than the running code, and `PRIMASK` = 0.

NVIC lines are **level-sensitive**: a handler that does not clear the device flag is re-entered forever, so it starts by **acknowledging at the device** (`CMSDK_UART0->INTSTATUS = UART_INT_TX;` , W1C).

6.3 Priorities, Preemption, and Nesting

When several exceptions compete, priorities decide which runs first and which can interrupt another.

Lower value means more urgent. Only a **strictly more urgent** exception preempts (nesting); others stay **pending**.

Exception	Priority
Faults	`0x00` or more urgent
SVCall	`0x80`

Exception	Priority
UART0 TX (IRQ 1)	`0xC0`
SysTick	`0xE0`

6.4 Races on Shared Data and Critical Sections

When `main` and a handler share data, an interrupt arriving in the middle of an update can corrupt it. A critical section prevents this by temporarily masking interrupts.

`volatile` forces real memory accesses but does not make an update indivisible:

```
/* count++ */
    ldr  r1, [r0]       /* r1 = count            (say 5)                      */
    adds r1, r1, #1     /* r1 = 6   <- the handler runs here, sets count = 6  */
    str  r1, [r0]       /* count = 6: the handler's update is lost            */
```

Operation on shared data	Safe?
Single aligned 32-bit read or write	yes
Read-modify-write (`++`, `+=`, `&=`, OR-assignment)	no: **lost update** (a **race**)
Update of several related variables	no

A **critical section** masks interrupts through `PRIMASK`, which C cannot name; two assembly functions provide nestable masking:

```
    .syntax unified
    .cpu cortex-m3
    .thumb
    .section .text,"ax",%progbits

/* uint32_t irq_save(void) */
    .global irq_save
    .type irq_save, %function
    .thumb_func
irq_save:
    mrs r0, primask             /* old = PRIMASK; r0 = return value */
    cpsid i                     /* PRIMASK = 1: interrupts masked */
    bx lr                       /* return old (in r0) */
    .size irq_save, . - irq_save

/* void irq_restore(uint32_t old) */
    .global irq_restore
    .type irq_restore, %function
    .thumb_func
```

```
irq_restore:
    msr primask, r0             /* PRIMASK = old (first argument, r0) */
    bx lr                       /* return */
    .size irq_restore, . - irq_restore
```

```
uint32_t a = irq_save();     /* PRIMASK 0 -> 1, a = 0 */
uint32_t b = irq_save();     /* PRIMASK already 1, b = 1 */
irq_restore( b );            /* PRIMASK stays 1: a plain cpsie here would unmask too early */
irq_restore( a );            /* PRIMASK back to 0 */
```

Rule	Reason
Restore the saved value, never `cpsie i`	sections **nest** correctly
Only the main code needs it	a handler cannot be interrupted by the main code
Short, never waiting	interrupts stay pending until `irq_restore`

6.5 Ring Buffer

A ring buffer is the data structure that connects the program (which produces bytes) and the interrupt handler (which consumes them).

A **ring buffer** is an array used as a FIFO: producer at `head`, consumer at `tail`, both wrapping.

A RING BUFFER

A **ring buffer** is an array used as a queue: the producer stores at `head`, the consumer takes at `tail`, and both indices wrap around to 0 at the end.

```
index:   0   1   2   3   4   5   6   7        SIZE = 8
       [ ] [ ] [a] [b] [c] [ ] [ ] [ ]
                ^tail      ^head             3 bytes waiting: a, b, c
```

```
next = ( i + 1 ) & ( SIZE - 1 );              /* = (i + 1) % SIZE when SIZE is a power of two */
empty:  head == tail
full:   ( ( head + 1 ) & ( SIZE - 1 ) ) == tail /* one slot always stays unused */
```

- A power-of-two size turns `% SIZE` into a cheap `& (SIZE - 1)` (no division)
- Each index has one writer: `head` only by the producer, `tail` only by the consumer

With SIZE = 64, how many bytes can wait at most, and why not 64? Answer: **63**. If all 64 slots could be used, a full buffer would also have `head == tail`, and "full" would look exactly like "empty".

A.A.2026-2027 | Week 3 - Interrupts & IO | Embedded Operating Systems and Architecture 62

Operation	Expression
Next index	`( i + 1 ) & ( SIZE - 1 )`, equal to `% SIZE` for a power of two, with no division
Empty	`head == tail`
Full	`( ( head + 1 ) & ( SIZE - 1 ) ) == tail`

Each index has one writer. One slot stays unused, so 64 entries hold **63** bytes: otherwise full and empty would both satisfy `head == tail`.

6.6 Interrupt-Driven Transmitter

The interrupt-driven UART driver combines the previous pieces: the ring buffer, the critical section, and the TX interrupt handler.

```
#define TX_BUF_SIZE 64                  /* a power of two */

static char tx_buf[TX_BUF_SIZE];        /* bytes waiting to be sent */
static volatile unsigned tx_head;       /* next free slot: written by uart_write */
static volatile unsigned tx_tail;       /* oldest byte: written by the handler (and priming) */
static volatile int tx_busy;            /* 1 while the UART sends and its IRQ will come */

static int tx_empty( void ) { return tx_head == tx_tail; }
static int tx_full( void )  { return ( ( tx_head + 1 ) & ( TX_BUF_SIZE - 1 ) ) == tx_tail; }

static char tx_pop( void )              /* take the oldest byte; buffer not empty */
{
    char c = tx_buf[tx_tail];
    tx_tail = ( tx_tail + 1 ) & ( TX_BUF_SIZE - 1 );
    return c;
}

void uart_init( void )
{
    CMSDK_UART0->BAUDDIV = 16;
    CMSDK_UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN;
    tx_head = 0;
    tx_tail = 0;
    tx_busy = 0;
    NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* below SVCall, above
SysTick */
    NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
}

unsigned uart_write( const char *buf, unsigned len )
{
    unsigned accepted = 0;
    uint32_t saved = irq_save();               /* the handler must not run during the updates
*/
```

```
    while ( accepted < len && !tx_full() )      /* copy what fits */
    {
        tx_buf[tx_head] = buf[accepted];
        tx_head = ( tx_head + 1 ) & ( TX_BUF_SIZE - 1 );
        accepted++;
    }

    if ( !tx_busy && !tx_empty() )              /* idle: no IRQ will come, so prime it */
    {
        CMSDK_UART0->CTRL |= UART_CTRL_TXIRQEN; /* FIRST enable the TX interrupt ... */
        while ( CMSDK_UART0->STATE & UART_STATE_TXBF )
        {
        }                                       /* (a polled byte may still be leaving) */
        CMSDK_UART0->DATA = (uint32_t) (unsigned char) tx_pop(); /* ... THEN write */
        tx_busy = 1;
    }

    irq_restore( saved );
    return accepted;
}

void UART0TX_Handler( void )
{
    CMSDK_UART0->INTSTATUS = UART_INT_TX;       /* acknowledge at the device (W1C) */

    if ( !tx_empty() )
    {
        CMSDK_UART0->DATA = (uint32_t) (unsigned char) tx_pop(); /* send the next byte */
    }
    else
    {
        CMSDK_UART0->CTRL &= ~UART_CTRL_TXIRQEN;  /* nothing left: interrupt off */
        tx_busy = 0;                              /* transmitter idle */
    }
}
```

Component	Role
`uart_write` (**producer**)	critical section; copies what fits and **returns the count accepted**, never waits; **primes** an idle transmitter with the first byte
`UART0TX_Handler` (**consumer**)	acknowledges, sends the next byte or disables `TXIRQEN`; no critical section needed

IMPORTANT

Priming order The TX interrupt fires when a byte has been sent **and** `TXIRQEN` was set when that byte was written. Enabling it while idle raises nothing; enabling it after the write misses that byte. Correct order: first `TXIRQEN`, then the first byte.

EXAMPLE

160 bytes through the 64-byte buffer `uart_write` is called 3 times (63, 63, 34 accepted); `UART0TX_Handler` runs **160 times**, the last one disabling `TXIRQEN`.

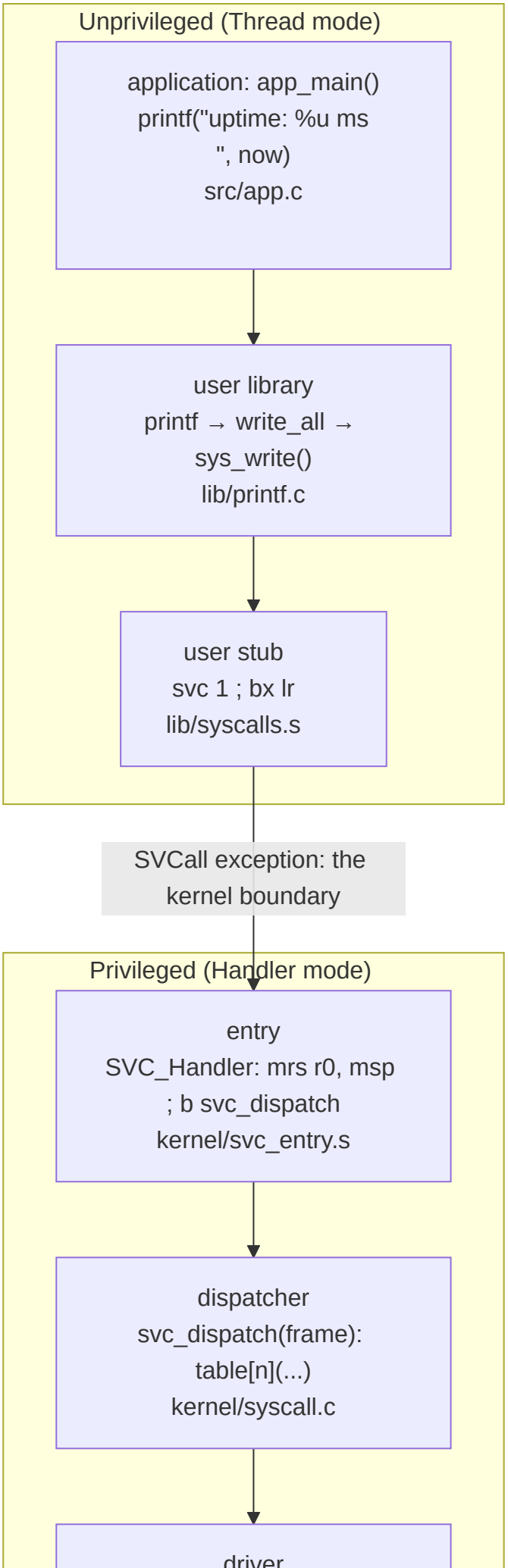
7 System Calls and the Kernel Boundary

Application code can be made unprivileged, so that it cannot reconfigure the machine. It then needs a controlled way to ask the kernel for services: the system call, implemented with the `svc` instruction.

7.1 Privilege Model and Layered Structure

The Cortex-M distinguishes **privileged** and **unprivileged** execution. The system is organized in layers, with the application on the unprivileged side and the kernel and drivers on the privileged side.

Mechanism	Behaviour
`CONTROL.nPRIV` = 1 in Thread mode	execution becomes unprivileged; one-way (unprivileged code cannot clear the bit)
Unprivileged access to the System Control Space (SysTick, NVIC, SCB)	**BusFault**
`svc #n`	allowed; raises **SVCall**, whose handler runs in Handler mode, always privileged
Return from SVCall	resumes after `svc`, still unprivileged


Unprivileged (Thread mode)
application: app_main()
printf("uptime: %u ms
", now)
src/app.c
user library
printf → write_all →
sys_write()
lib/printf.c
user stub
svc 1 ; bx lr
lib/syscalls.s
SVCall exception: the
kernel boundary
Privileged (Handler mode)
entry
SVC_Handler: mrs r0, msp
; b svc_dispatch
kernel/svc_entry.s
dispatcher
svc_dispatch(frame):
table[n](...)
kernel/syscall.c
driver

Without an MPU the UART stays reachable from unprivileged code, so "all I/O through system calls" is a **design choice**, with one hard reason: `cpsid i` **is silently ignored unprivileged**, so `uart_write` is correct only when privileged.

7.2 The System-Call ABI

Caller and kernel must agree on how a request is expressed at the machine level: which service is requested, where the arguments are, and where the result goes.

The **ABI** (Application Binary Interface): service number in the immediate of `svc #n`, arguments in `r0`–`r2`, result in `r0`.

Number	Service	Arguments	Result
1	`sys_write`	`buf`, `len`	bytes accepted, 0 to `len` (`uart_write`)
2	`sys_read`	none	one received byte, or `-1` (`uart_getc_polled`)
3	`sys_uptime`	none	milliseconds since start (`systick_uptime_ms`)
0, ≥ 4	none	none	`-1`

No service ever waits: the caller handles partial results.

7.3 Encoding of `svc #n` and Decoding from the Stacked Frame

The service number is not passed in a register but encoded inside the `svc` instruction itself; the kernel reads it, together with the arguments, from the stacked frame.

```
$ arm-none-eabi-objdump -d build/scratch-os.axf          one instruction per line
000008e0 <sys_write>:
 8e0:  df01   svc  1
 8e2:  4770   bx   lr

$ arm-none-eabi-objdump -s -j .text build/scratch-os.axf  raw bytes, memory order
 08e0 01df7047 02df7047 03df7047
```

`svc_1` 16-bit **= 0xDF01**

Thumb encoding of svc #1 (one halfword)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPCODE 0xDF								IMM8 0x1							
1	1	0	1	1	1	1	1	0	0	0	0	0	0	0	1

Bits	Field	Value	Description
15:8	`OPCODE`	0xDF (223)	0xDF identifies svc
7:0	`IMM8`	0x1 (1)	Service number n

Memory is **little-endian**: `objdump -d` shows the halfword `df01`, `objdump -s` the bytes `01 df`. With `uint32_t`

`*frame` pointing at the stacked frame (4.3, `frame[6]` = `pc`):

Item	Location	Reason
Service number `n`	`( ( uint8_t * ) frame[6] )[-2]`	the stacked `pc` points **after** the 2-byte `svc`
Arguments	`frame[0]` – `frame[2]`	stacked by the hardware: no copy needed
Result	`frame[0]`	the exception return pops it into the caller's `r0`

7.4 Assembly Entry Point `SVC_Handler`

To read the stacked frame, the kernel must know where it is at the very first instruction of the handler; this requires a short entry point in assembly.

The kernel needs **MSP** at entry to find the frame, but a C prologue moves `sp` first:

```
0000027a <svc_dispatch>:
 27a:  b580   push  {r7, lr}     @ sp = sp - 8
 27c:  b084   sub   sp, #16      @ sp = sp - 16
 27e:  af00   add   r7, sp, #0
```

`sp` has moved by a compiler-dependent 24 bytes. A handler that must find the frame needs an assembly entry:

```
    .global SVC_Handler         /* replaces the weak alias in the vector table */
    .type SVC_Handler, %function
    .thumb_func
SVC_Handler:
    mrs r0, msp                 /* r0 = MSP: frame = address of the 8 saved registers */
    b svc_dispatch              /* svc_dispatch(frame); lr still = EXC_RETURN */
    .size SVC_Handler, . - SVC_Handler
```

Instruction	Purpose
`mrs r0, msp`	frame address in `r0`, the first AAPCS argument of `svc_dispatch(uint32_t *frame)`
`b` (not `bl`)	**tail branch**: `bl` would overwrite `lr` and lose `EXC_RETURN`; with `b`, the normal return of `svc_dispatch` through `lr` performs the exception return directly

7.5 Dispatch Table and User-Side Stubs

On the kernel side, a table maps each service number to its implementation; on the application side, tiny assembly stubs turn a C call into an `svc`.

```
/* A service: up to 3 arguments (caller's r0-r2), one result (caller's r0). */
typedef uint32_t ( *syscall_fn )( uint32_t a0, uint32_t a1, uint32_t a2 );
```

```c
static uint32_t sys_write_impl( uint32_t a0, uint32_t a1, uint32_t a2 )
{
    (void) a2;
    return uart_write( ( const char * ) a0, a1 );   /* bytes accepted, 0..len */
}

static uint32_t sys_read_impl( uint32_t a0, uint32_t a1, uint32_t a2 )
{
    (void) a0; (void) a1; (void) a2;
    return ( uint32_t ) uart_getc_polled();         /* byte, or -1 */
}

static uint32_t sys_uptime_impl( uint32_t a0, uint32_t a1, uint32_t a2 )
{
    (void) a0; (void) a1; (void) a2;
    return systick_uptime_ms();
}

static const syscall_fn table[] = {             /* entry 0 stays empty (0) */
    [SYS_WRITE]  = sys_write_impl,
    [SYS_READ]   = sys_read_impl,
    [SYS_UPTIME] = sys_uptime_impl,
};

#define TABLE_SIZE ( sizeof( table ) / sizeof( table[0] ) )

void svc_dispatch( uint32_t *frame )
{
    uint8_t n = ( ( uint8_t * ) frame[6] )[-2];  /* low byte of svc #n before the stacked pc
*/

    if ( n >= TABLE_SIZE || table[n] == 0 )
    {
        frame[0] = ( uint32_t ) -1;             /* unknown service */
        return;
    }
    frame[0] = table[n]( frame[0], frame[1], frame[2] );  /* result into the stacked r0 */
}
```

Element	Meaning
`syscall_fn`	one signature for all services, so they fit one array
Designated initializers	each entry at its number; entry 0 stays empty; `TABLE_SIZE` follows automatically

Element	Meaning
Bounds and null check	unknown service answers `-1`

```
/* int sys_write(const char *buf, unsigned len): buf in r0, len in r1 */
    .global sys_write
    .type sys_write, %function
    .thumb_func
sys_write:
    svc #1                      /* svc(SYS_WRITE, buf, len) */
    bx lr                       /* return r0 (set by the kernel) */
    .size sys_write, . - sys_write
```

`sys_read` (`svc #2`) and `sys_uptime` (`svc #3`) are identical. The number is part of the encoding, so it is a literal kept in sync by hand with the C header. In gdb, `x/2bx ((unsigned *)$sp)[6] - 2` at `SVC_Handler` shows `0x03 0xdf` (`svc #3`), and `finish` from `svc_dispatch` lands on the stub's `bx lr` : the return was the exception return.

7.6 User-Space `printf` , `write_all` , and `sleep_ms`

Unprivileged code cannot use `kprintf` , so the application gets its own `printf` , built only on system calls.

Aspect	`kprintf` (kernel)	`printf` (user)
Each character goes to	`uart_putc_polled`	a 64-byte buffer on the stack
When the buffer is full, or at the end	—	`write_all(buf, used)`
Path to the UART	direct register access (privileged)	`sys_write` → `svc #1`

```
#define PRINTF_BUF_SIZE 64      /* characters per sys_write */

void write_all( const char *buf, unsigned len )
{
    while ( len > 0 )
    {
        int n = sys_write( buf, len );  /* the kernel takes what fits */
        buf += n;
        len -= n;
        if ( n == 0 )
        {
            __asm volatile ( "wfi" );   /* full: sleep until an interrupt */
        }
    }
}
```

```
struct printf_ctx
{
    char buf[PRINTF_BUF_SIZE];  /* characters not sent yet */
    unsigned used;              /* how many of buf are in use */
    int total;                  /* characters produced so far */
};

static void printf_out( char c, void *ctx )    /* store one char, flush when full */
{
    struct printf_ctx *p = ctx;
    p->buf[p->used] = c;
    p->used++;
    p->total++;
    if ( p->used == PRINTF_BUF_SIZE )
    {
        write_all( p->buf, p->used );
        p->used = 0;
    }
}

int printf( const char *fmt, ... )
{
    struct printf_ctx p;                    /* on the stack, about 72 bytes */
    p.used = 0;
    p.total = 0;
    va_list ap;
    va_start( ap, fmt );
    vformat( printf_out, &p, fmt, ap );
    va_end( ap );
    write_all( p.buf, p.used );             /* send what is left */
    return p.total;
}

void sleep_ms( uint32_t ms )
{
    uint32_t start = sys_uptime();
    while ( sys_uptime() - start < ms )     /* safe across wraparound */
    {
        __asm volatile ( "wfi" );
    }
}
```

Element	Design point
64-byte buffer	one system call per 64 characters
`write_all`	handles **partial writes**; on 0 accepted, `wfi` (allowed unprivileged) until the TX interrupt frees room
`sleep_ms`	the **waiting happens in Thread mode**, where interrupts run

> **WARNING**
>
> Hidden `memset` in a freestanding build `struct printf_ctx p = { .used = 0, .total = 0 };` fails to link (`undefined reference to 'memset'`): GCC zero-fills the array by calling `memset` , absent without a C library.

7.7 Kernel Start and Unprivileged Application

`main` initializes the system while still privileged, then drops privilege and hands control to the application, which from then on uses only system calls.

```
int main( void )
{
    uart_init();                                /* 1. console first */
    kprintf( "scratch-os: kernel started\n" );  /* 2. polled print */
    systick_init();                             /* 3. 10 ms tick */
    kprintf( "dropping to unprivileged mode\n" );
    __asm volatile (            /* 4. CONTROL = CONTROL | 1; (nPRIV = 1) */
        "mrs r0, control    \n" /* r0 = CONTROL */
        "orr r0, r0, #1     \n" /* r0 |= 1: set nPRIV */
        "msr control, r0    \n" /* CONTROL = r0: now unprivileged */
        "isb                \n" /* refetch under the new rules */
        : : : "r0", "memory" );
    app_main();                                 /* 5. never returns */
    for ( ;; ) { }                              /* not reached */
}

void app_main( void )
{
    printf( "app: hello from unprivileged code\n" );
    uint32_t last = 0;                          /* when the uptime was last printed */

    for ( ;; )
    {
        uint32_t now = sys_uptime();
        if ( now - last >= 1000 )               /* one second passed? */
        {
```

```
            printf( "uptime: %u ms\n", now );
            last += 1000;
        }
        int c = sys_read();                    /* never waits: -1 = no key */
        if ( c != -1 )
        {
            printf( "key: %c\n", c );
        }
        sleep_ms( 10 );
    }
}
```

Element	Reason
Order: console, tick, drop	printing first; the tick serves `sys_uptime` and `sleep_ms`
`isb`	discards instructions fetched under privileged rules
Clock comparison	counting iterations would drift

```
scratch-os: kernel started
dropping to unprivileged mode
app: hello from unprivileged code
uptime: 1000 ms
key: a
key: b
uptime: 2000 ms
```

The first two lines come from polled `kprintf`, the rest through `printf` → `sys_write` → `svc #1` → `uart_write` → `UART0TX_Handler`. In gdb, `$control` is `0x1` even inside `sys_write_impl` (privileged only because in Handler mode), and `$xpsr & 0x1ff` = 11 (SVCall).

7.8 Kernel Rules: No `svc` Inside the Kernel, Never Wait

Two rules keep the kernel correct; breaking either one stops the system, as the following failures show.

No system calls in kernel code. An `svc` is **synchronous** and cannot stay pending: if not more urgent than the running code, it **escalates** to HardFault (HFSR bit 30, **FORCED**, e.g. `HFSR=0x40000000`).

Where `svc` executes	Current priority	Result
`app_main`, `main` (Thread mode)	none (lowest)	taken
`SysTick_Handler` / `UART0TX_Handler`	`0xE0` / `0xC0`	taken: SVCall is more urgent and nests
`SVC_Handler` and anything it calls	`0x80`	HardFault
A fault handler	`0x00` or more urgent	HardFault

Where `svc` executes	Current priority	Result
Between `irq_save` and `irq_restore`	masked (`PRIMASK` = 1)	HardFault

The kernel never waits. If `sys_write_impl` loops until all bytes are accepted, a 130-byte write prints one byte and freezes: only `UART0TX_Handler` (`0xC0`) can drain the buffer, but the CPU is in SVCall (`0x80`), more urgent, so TX and SysTick (`0xE0`) stay pending forever. Hence `sys_write` returns the accepted count and `write_all` waits in **Thread mode**.

> **IMPORTANT**
>
> Two kernel invariants Never block inside a system call (partial results, waiting in user space). Never issue a system call from the kernel (output via polled `kprintf`).